Ansible / RHCE Complete Tutorial

8 Modules — From Zero to RHCE EX294

Module	Title	Topics Covered
Module 1	Foundations	Inventory, ad-hoc commands, YAML, first playbook
Module 2	Playbooks In Depth	Variables, handlers, when, loop, register
Module 3	Ansible Modules	dnf, service, file, copy, template, user, firewalld
Module 4	Variables & Facts	Facts, set_fact, hostvars, group_vars, filters
Module 5	Roles	Role structure, defaults vs vars, Galaxy, site.yml
Module 6	Jinja2 Templates	Loops, conditionals, macros, filters, real configs
Module 7	Advanced Topics	Vault, tags, block/rescue, delegate_to, serial, async
Module 8	RHCE Exam Projects	5 complete exam-style automation scenarios

Multi-VM Lab Setup • RHCE EX294 Exam Ready • Production Patterns

Module 1 Ansible Foundations

Inventory · Ad-hoc commands · YAML · First playbook

What Ansible Actually Is

Before touching any commands, you need the right mental model. Ansible is not a programming language and it's not a script runner. It's a declarative configuration management tool — you describe the desired state of your servers, and Ansible figures out how to get there.

The key difference from Bash scripts: Ansible is idempotent. Run the same playbook 10 times — it only makes changes when something is actually wrong. Bash scripts run every command every time.

Concept	Bash/Python way	Ansible way
Install nginx	ssh server 'yum install nginx'	- package: name=nginx state=present
Start a service	ssh server 'systemctl start nginx'	- service: name=nginx state=started
Copy a file	scp file server:/etc/	- copy: src=file dest=/etc/

The Big Picture — How Ansible Works

```
Your Laptop / Control Node

        ■

        ■ SSH (no agent needed on managed nodes)

        ■

    ■■■■■▼■■■■■ ■■■■■■■■■■■ ■■■■■■■■■■■

    ■ web01 ■ ■ web02 ■ ■ db01 ■

    ■ RHEL 9 ■ ■ RHEL 9 ■ ■ RHEL 9 ■

    ■■■■■■■■■■■ ■■■■■■■■■■■ ■■■■■■■■■■■

    Managed Nodes — only need Python + SSH
```

Three things Ansible needs:

- Inventory — list of servers to manage
- Playbook — what to do on those servers
- SSH access — key-based auth to all managed nodes

1.1 Lab Setup — Multiple VMs

```
control 192.168.56.10 — where you run ansible commands

web01 192.168.56.11 — managed node

web02 192.168.56.12 — managed node

db01 192.168.56.13 — managed node
```

Step 1 — Install Ansible on control node only

```
# RHEL / Rocky / AlmaLinux

sudo dnf install -y ansible-core
```

```
# Or via pip (gets latest version)
pip3 install ansible --break-system-packages

# Verify
ansible --version
```

Step 2 — Set up SSH key-based auth

```
# Generate key on control node
ssh-keygen -t ed25519 -C "ansible-control"

# Copy key to each managed node
ssh-copy-id ansible@web01
ssh-copy-id ansible@web02
ssh-copy-id ansible@db01

# Test — should NOT ask for password
ssh ansible@web01 "hostname"
ssh ansible@web02 "hostname"
ssh ansible@db01 "hostname"
```

Step 3 — Create ansible user on managed nodes

```
# Run on each managed node
sudo useradd -m -s /bin/bash ansible
sudo echo "ansible ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/ansible
sudo chmod 440 /etc/sudoers.d/ansible
```

1.2 Ansible Configuration File

```
# Create your project directory
mkdir ~/ansible-lab
cd ~/ansible-lab

# Create ansible.cfg
cat > ansible.cfg << 'EOF'
[defaults]
inventory = inventory.ini # default inventory file
remote_user = ansible # user to SSH as
private_key_file = ~/.ssh/id_ed25519 # SSH key to use
```

```
host_key_checking = False # skip host key prompts in lab

forks = 5 # run on 5 hosts in parallel

[privilege_escalation]

become = True # use sudo by default

become_method = sudo

become_user = root

EOF
```

> ■ ansible.cfg lookup order: (1) ANSIBLE_CONFIG env var, (2) ./ansible.cfg current directory ← always use this in projects, (3) ~/.ansible.cfg user home, (4) /etc/ansible/ansible.cfg system default

1.3 Inventory — Your Server List

```
# inventory.ini — INI format (most common)

# ■■ Groups ■■■
```

```
# List all hosts
ansible all --list-hosts

# List hosts in a group
ansible webservers --list-hosts

# Show as a graph
ansible-inventory --graph
```

```
@all:
  |--@production:
  |  |--@webservers:
  |  |  |--web01
  |  |  |--web02
  |  |--@databases:
  |  |  |--db01
```

1.4 Ad-Hoc Commands

Ad-hoc commands run a single task directly from the command line without writing a playbook. Perfect for quick checks and one-off tasks.

```
# Syntax: ansible <host-pattern> -m <module> -a "<arguments>"

# Ping all hosts (tests SSH connectivity)
ansible all -m ping

# Run a shell command on all hosts
ansible all -m shell -a "uptime"
ansible all -m shell -a "df -h /"
ansible all -m shell -a "cat /etc/redhat-release"

# Target specific groups
ansible webservers -m shell -a "systemctl status nginx"
ansible databases -m shell -a "systemctl status postgresql"

# Gather facts about hosts
ansible web01 -m setup
ansible web01 -m setup -a "filter=ansible_os_family"
ansible web01 -m setup -a "filter=ansible_memory_mb"

# Manage packages ad-hoc
```

```
ansible webservers -m dnf -a "name=nginx state=present"

ansible webservers -m dnf -a "name=nginx state=absent"


# Manage services ad-hoc

ansible webservers -m service -a "name=nginx state=started enabled=yes"


# Create a user ad-hoc

ansible all -m user -a "name=john state=present shell=/bin/bash"
```

1.5 Your First Playbook — YAML Syntax

```
# first_playbook.yml

--- # YAML document start marker

- name: My first play # Play name

  hosts: webservers # Which hosts/groups to target

  become: true # Use sudo for all tasks


  tasks:


    - name: Install nginx

      ansible.builtin.dnf:

        name: nginx

        state: present


    - name: Start and enable nginx

      ansible.builtin.service:

        name: nginx

        state: started

        enabled: true


    - name: Create a test file

      ansible.builtin.copy:

        content: "Hello from Ansible!\n"

        dest: /var/www/html/index.html

        owner: nginx

        group: nginx

        mode: '0644'
```

```
# Run the playbook
```

```
ansible-playbook first_playbook.yml

# Dry run — shows what WOULD change without making changes
ansible-playbook first_playbook.yml --check

# Verbose output
ansible-playbook first_playbook.yml -v # basic verbose
ansible-playbook first_playbook.yml -vv # more detail
ansible-playbook first_playbook.yml -vvv # connection debug

# Limit to specific hosts
ansible-playbook first_playbook.yml --limit web01

# List tasks without running them
ansible-playbook first_playbook.yml --list-tasks
```

Understanding Playbook Output

```
PLAY [My first play] ************************************

TASK [Gathering Facts] **********************************
ok: [web01]
ok: [web02]

TASK [Install nginx] ************************************
changed: [web01]          ← made a change
changed: [web02]          ← made a change

TASK [Start and enable nginx] ***************************
ok: [web01]               ← already in desired state
ok: [web02]               ← already in desired state

PLAY RECAP **********************************************
web01 : ok=3  changed=1  unreachable=0  failed=0
web02 : ok=3  changed=1  unreachable=0  failed=0
```

Status	Meaning
ok	Task ran, no change needed — already in desired state
changed	Task ran and made a change
failed	Task failed
unreachable	Could not connect to host
skipped	Task was skipped — condition was false

1.6 YAML — The Language of Playbooks

```yaml
# Strings
name: nginx # unquoted string
description: "my web server" # quoted string
path: '/etc/nginx/nginx.conf' # single quotes

# Numbers and booleans
port: 80
enabled: true # or yes, True, TRUE
disabled: false # or no, False, FALSE

# Lists
packages:
  - nginx
  - httpd
  - php-fpm

# Inline list
packages: [nginx, httpd, php-fpm]

# Dictionaries
webserver:
  name: nginx
  port: 80
  enabled: true

# Multiline strings — | preserves newlines
script: |
  #!/bin/bash
  echo "hello"
  systemctl restart nginx
```

> ■ YAML INDENTATION: Always 2 spaces — NEVER tabs. A tab character will break your playbook with a cryptic error.

1.7 Real-World Example — Multi-Server Web Setup

```yaml
# web_setup.yml
---
```

```yaml
- name: Configure web servers
  hosts: webservers
  become: true

  vars:
    web_root: /var/www/html
    packages:
      - nginx
      - php-fpm
      - git

  tasks:

    - name: Install required packages
      ansible.builtin.dnf:
        name: "{{ packages }}"
        state: present

    - name: Ensure nginx is started and enabled
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true

    - name: Deploy index page
      ansible.builtin.copy:
        content: |
          <html><body><h1>Managed by Ansible</h1></body></html>
        dest: "{{ web_root }}/index.html"
        owner: nginx
        group: nginx
        mode: '0644'

    - name: Open firewall for HTTP
      ansible.builtin.firewalld:
        service: http
        state: enabled
```

```yaml
        permanent: true
        immediate: true

- name: Configure database server
  hosts: databases
  become: true

  tasks:

    - name: Install PostgreSQL
      ansible.builtin.dnf:
        name:
          - postgresql-server
          - postgresql-contrib
        state: present

    - name: Initialize PostgreSQL database
      ansible.builtin.command:
        cmd: postgresql-setup --initdb
        creates: /var/lib/pgsql/data/postgresql.conf

    - name: Start and enable PostgreSQL
      ansible.builtin.service:
        name: postgresql
        state: started
        enabled: true
```

■ Key Takeaways

- Ansible is agentless — managed nodes only need SSH + Python
- Idempotent — safe to run repeatedly, only changes what needs changing
- ansible.cfg in your project directory controls all settings
- inventory.ini groups your servers — [webservers], [databases], [prod:children]
- Ad-hoc: ansible all -m ping, ansible webservers -m shell -a 'uptime'
- Playbooks are YAML — always 2-space indent, never tabs
- ok = no change needed, changed = Ansible made a change, failed = problem
- --check for dry run, -v for verbose, --limit to target specific hosts

■ Module 1 Exercise

Set up your lab and write a playbook lab_setup.yml that:

- Targets all hosts in your inventory
- Installs: vim, curl, wget, net-tools, bash-completion on all hosts

- Sets the timezone to UTC using community.general.timezone
- Creates directory /opt/ansible-managed on all hosts
- Creates /opt/ansible-managed/info.txt with content: 'Managed by Ansible — hostname: <actual hostname>'
- Runs a second play targeting only webservers that installs and starts nginx
- Run with --check first, then for real, then again to confirm idempotency

Module 2 Playbooks In Depth

Variables · Handlers · Conditionals · Loops · Register

■ Module 1 Solution — lab_setup.yml

```yaml
---

# Play 1: All Hosts

- name: Base setup for all managed hosts

  hosts: all

  become: true


  vars:

    base_packages:

      - vim

      - curl

      - wget

      - net-tools

      - bash-completion

    managed_dir: /opt/ansible-managed


  tasks:


    - name: Install base packages on all hosts

      ansible.builtin.dnf:

        name: "{{ base_packages }}"

        state: present


    - name: Set timezone to UTC

      community.general.timezone:

        name: UTC


    - name: Create managed directory

      ansible.builtin.file:

        path: "{{ managed_dir }}"

        state: directory

        owner: root

        group: root
```

```
        mode: '0755'

    - name: Deploy host info file
      ansible.builtin.copy:
        content: "Managed by Ansible — hostname: {{ ansible_hostname }}\n"
        dest: "{{ managed_dir }}/info.txt"
        mode: '0644'

# Play 2: Web Servers Only
- name: Web server setup
  hosts: webservers
  become: true

  tasks:

    - name: Install nginx
      ansible.builtin.dnf:
        name: nginx
        state: present

    - name: Start and enable nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```

> ■ ansible_hostname is an Ansible fact — automatically collected from each managed node. The {{ }} syntax is Jinja2 templating used throughout Ansible.

1. Variables — Three Ways to Define Them

```
# Method 1: vars block in the play
- name: Configure web server
  hosts: webservers
  vars:
    http_port: 80
    web_root: /var/www/html
    packages:
```

```
      - nginx
      - php-fpm

# Method 2: vars_files — load from external YAML file
- name: Configure web server
  hosts: webservers
  vars_files:
    - vars/web_vars.yml
    - vars/common.yml

# vars/web_vars.yml contents:
# http_port: 80
# web_root: /var/www/html
```

```
# Method 3: Pass at runtime with -e
ansible-playbook site.yml -e "http_port=8080"
ansible-playbook site.yml -e "env=production db_host=db01"
ansible-playbook site.yml -e @extra_vars.yml # from file
```

Precedence	Source	Notes
1 (HIGHEST)	-e (command line)	Always wins
2	task vars	
3	play vars	
4	host_vars/hostname.yml	
5	group_vars/groupname.yml	
6	group_vars/all.yml	
7 (LOWEST)	inventory vars	

group_vars and host_vars — The Right Way

```
ansible-lab/
■■■ ansible.cfg
■■■ inventory.ini
■■■ group_vars/
■ ■■■ all.yml ← applies to ALL hosts
■ ■■■ webservers.yml ← applies to webservers group only
■ ■■■ databases.yml ← applies to databases group only
■■■ host_vars/
■ ■■■ web01.yml ← applies to web01 only
```

```
■ ■■■ db01.yml

■■■ site.yml
```

```yaml
# group_vars/all.yml — variables for every host

---

managed_dir: /opt/ansible-managed

ntp_servers:

   - 0.rhel.pool.ntp.org

   - 1.rhel.pool.ntp.org

admin_email: admin@company.com


# group_vars/webservers.yml

---

http_port: 80

https_port: 443

web_root: /var/www/html


# host_vars/web01.yml — override just for web01

---

http_port: 8080 # this host uses a non-standard port

server_role: primary
```

2. Handlers — Run Tasks Only When Something Changes

Handlers are tasks that only run when notified by another task. The classic use case is restarting a service only when its config file changes — not every time the playbook runs.

```yaml
---

- name: Configure nginx

   hosts: webservers

   become: true


   tasks:


     - name: Install nginx

       ansible.builtin.dnf:

         name: nginx

         state: present


     - name: Deploy nginx config
```

```yaml
      ansible.builtin.copy:
        src: files/nginx.conf
        dest: /etc/nginx/nginx.conf
      notify: Restart nginx # triggers handler IF this task changed

    - name: Deploy virtual host config
      ansible.builtin.template:
        src: templates/vhost.conf.j2
        dest: /etc/nginx/conf.d/site.conf
      notify: Reload nginx

    - name: Ensure nginx is started
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true

  handlers:

    - name: Restart nginx # must match notify: string exactly
      ansible.builtin.service:
        name: nginx
        state: restarted

    - name: Reload nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
```

> ■ Handlers only run at the END of the play. If a task notifies a handler but nothing actually changed (ok not changed), the handler does NOT run. If the same handler is notified multiple times, it only runs ONCE.

3. Conditionals — when:

```yaml
  tasks:

    # Run only on RedHat-family systems
    - name: Install nginx (RedHat)
```

```yaml
    ansible.builtin.dnf:
      name: nginx
      state: present
    when: ansible_os_family == "RedHat"

  # Run only on RHEL 9
  - name: Enable RHEL 9 specific repo
    ansible.builtin.command: dnf config-manager --enable rhel-9-extras
    when:
      - ansible_distribution == "RedHat"
      - ansible_distribution_major_version == "9"

  # Run when variable is defined
  - name: Configure custom port
    ansible.builtin.lineinfile:
      path: /etc/myapp/config
      line: "port={{ custom_port }}"
    when: custom_port is defined

  # Run when variable matches a value
  - name: Deploy production config
    ansible.builtin.copy:
      src: files/prod.conf
      dest: /etc/myapp/config
    when: env == "production"
```

Pattern	Example
String equality	when: ansible_os_family == "RedHat"
Numeric comparison	when: ansible_memtotal_mb >= 2048
Check if in list	when: "'nginx' in ansible_facts.packages"
Variable exists	when: my_variable is defined
Variable missing	when: my_variable is not defined
Target specific host	when: inventory_hostname == "web01"
AND — all must be true	when: [condition1, condition2]
OR	when: condition1 or condition2

4. Loops — loop:

```yaml
tasks:

  # Loop over a list
  - name: Install multiple packages
    ansible.builtin.dnf:
      name: "{{ item }}"
      state: present
    loop:
      - vim
      - curl
      - wget
      - git

  # Loop over a list of dicts
  - name: Create users with full details
    ansible.builtin.user:
      name: "{{ item.name }}"
      uid: "{{ item.uid }}"
      groups: "{{ item.groups }}"
      shell: /bin/bash
      state: present
    loop:
      - { name: alice, uid: 1001, groups: "wheel,developers" }
      - { name: bob, uid: 1002, groups: "developers" }
      - { name: charlie, uid: 1003, groups: "qa" }

  # Loop with index
  - name: Create numbered files
    ansible.builtin.copy:
      content: "File number {{ idx + 1 }}\n"
      dest: "/tmp/file_{{ idx + 1 }}.txt"
    loop: "{{ range(1, 6) | list }}"
    loop_control:
      index_var: idx
      label: "file {{ idx + 1 }}" # cleaner output
```

5. register — Capture Task Output

```yaml
tasks:

  # Capture command output
  - name: Get disk usage
    ansible.builtin.shell: df -h / | awk 'NR==2 {print $5}'
    register: disk_usage

  - name: Show disk usage
    ansible.builtin.debug:
      msg: "Root disk usage: {{ disk_usage.stdout }}"

  # Use register in a conditional
  - name: Check if config file exists
    ansible.builtin.stat:
      path: /etc/myapp/config
    register: config_stat

  - name: Create config if missing
    ansible.builtin.copy:
      src: files/default.conf
      dest: /etc/myapp/config
    when: not config_stat.stat.exists

  # Loop and register
  - name: Check multiple services
    ansible.builtin.shell: systemctl is-active {{ item }}
    register: service_checks
    loop:
      - nginx
      - sshd
      - crond
    ignore_errors: true

  - name: Show service statuses
    ansible.builtin.debug:
      msg: "{{ item.item }}: {{ item.stdout }}"
```

```
    loop: "{{ service_checks.results }}"
```

Field	Contents
register_var.stdout	Command output as string
register_var.stderr	Error output as string
register_var.rc	Return code (0 = success)
register_var.stdout_lines	Output as list of lines
register_var.changed	True if task made a change
register_var.failed	True if task failed
register_var.stat.exists	True if file exists (stat module)
register_var.results	List of results when used with loop

6. debug — Print Variables During a Run

```
tasks:

  - name: Show hostname
    ansible.builtin.debug:
      msg: "Running on {{ ansible_hostname }}"

  - name: Show registered output
    ansible.builtin.debug:
      var: disk_usage.stdout

  - name: Show system info
    ansible.builtin.debug:
      msg: >
        Host: {{ ansible_hostname }}
        OS: {{ ansible_distribution }} {{ ansible_distribution_version }}
        RAM: {{ ansible_memtotal_mb }}MB

  # Show only when verbose (-v)
  - name: Debug only in verbose mode
    ansible.builtin.debug:
      msg: "Detail: {{ some_var }}"
      verbosity: 1 # only shows with -v or higher
```

7. ignore_errors and failed_when / changed_when

```
tasks:
```

```
    - name: Try to stop service that might not exist
      ansible.builtin.service:
        name: old-service
        state: stopped
      ignore_errors: true

    # Custom failure condition
    - name: Check disk space
      ansible.builtin.shell: df / | awk 'NR==2 {print $5}' | tr -d '%'
      register: disk_pct
      failed_when: disk_pct.stdout | int > 90

    # Custom changed condition
    - name: Run script
      ansible.builtin.shell: /opt/scripts/check.sh
      register: result
      changed_when: "'Updated' in result.stdout"
      failed_when: result.rc not in [0, 1] # 0 or 1 are both OK
```

8. Real-World Playbook — Server Hardening

```
# hardening.yml
---
- name: Security hardening for all servers
  hosts: all
  become: true

  vars:
    allowed_users:
      - ansible
      - admin

  tasks:

    - name: Install security packages
      ansible.builtin.dnf:
        name:
```

```yaml
        - fail2ban
        - aide
        - audispd-plugins
      state: present

    - name: Harden SSH configuration
      ansible.builtin.lineinfile:
        path: /etc/ssh/sshd_config
        regexp: "{{ item.regexp }}"
        line: "{{ item.line }}"
        state: present
        backup: true
      loop:
        - { regexp: '^#?PermitRootLogin', line: 'PermitRootLogin no' }
        - { regexp: '^#?PasswordAuthentication', line: 'PasswordAuthentication no' }
        - { regexp: '^#?MaxAuthTries', line: 'MaxAuthTries 3' }
        - { regexp: '^#?ClientAliveInterval', line: 'ClientAliveInterval 300' }
      notify: Restart sshd

    - name: Enable and start fail2ban
      ansible.builtin.service:
        name: fail2ban
        state: started
        enabled: true

    - name: Disable unused services
      ansible.builtin.service:
        name: "{{ item }}"
        state: stopped
        enabled: false
      loop:
        - bluetooth
        - cups
        - avahi-daemon
      ignore_errors: true
```

```yaml
  handlers:

    - name: Restart sshd
      ansible.builtin.service:
        name: sshd
        state: restarted
```

■ Key Takeaways

- vars: in the play, group_vars/, host_vars/ — use directories in real projects
- -e 'var=value' at runtime always wins — highest precedence
- Handlers only run when notified AND when the notifying task changed
- when: takes Jinja2 expressions — use facts like ansible_os_family
- loop: replaces repeating the same task — use {{ item }} to reference each value
- register: captures task output — use .stdout, .rc, .stat.exists, .results
- ignore_errors: true prevents a task failure from stopping the play
- changed_when: and failed_when: let you define what counts as changed or failed

■ Module 2 Exercise — user_setup.yml

Write a playbook user_setup.yml that:

- Defines dev_users (3 users with name, uid, comment) and admin_users (2 users) in group_vars/all.yml
- Uses loop: to create all dev_users with home directories and bash shell
- Uses loop: to create all admin_users and add them to the wheel group
- Uses register: to capture the result of getent passwd for one user
- Uses debug: to print whether that user was found
- Uses when: to only set password expiry policy on hosts with ansible_memtotal_mb > 512
- Has a handler that sends a debug message 'User setup complete'

Module 2 Solution — user_setup.yml

```yaml
# group_vars/all.yml
---
dev_users:
  - name: alice
    uid: 2001
    comment: "Alice Developer"
  - name: bob
    uid: 2002
    comment: "Bob Developer"
  - name: charlie
    uid: 2003
    comment: "Charlie Developer"
```

```yaml
admin_users:
  - name: sarah
    uid: 2010
    comment: "Sarah Admin"
  - name: mike
    uid: 2011
    comment: "Mike Admin"
```

```yaml
# user_setup.yml
---
- name: User provisioning across all hosts
  hosts: all
  become: true

  tasks:

    - name: Create developer users
      ansible.builtin.user:
        name: "{{ item.name }}"
        uid: "{{ item.uid }}"
        comment: "{{ item.comment }}"
        shell: /bin/bash
        create_home: true
        state: present
      loop: "{{ dev_users }}"
      loop_control:
        label: "{{ item.name }}"
      notify: User setup complete

    - name: Create admin users with sudo access
      ansible.builtin.user:
        name: "{{ item.name }}"
        uid: "{{ item.uid }}"
        comment: "{{ item.comment }}"
        shell: /bin/bash
        create_home: true
```

```
        groups: wheel
        append: true # add to wheel, don't replace existing groups
        state: present
      loop: "{{ admin_users }}"
      loop_control:
        label: "{{ item.name }}"
      notify: User setup complete

    - name: Check alice exists in passwd database
      ansible.builtin.shell: getent passwd alice
      register: alice_check
      ignore_errors: true
      changed_when: false # just reading — never a change

    - name: Report alice status
      ansible.builtin.debug:
        msg: >-
          {{ 'alice FOUND: ' + alice_check.stdout
             if alice_check.rc == 0
             else 'alice NOT found in passwd database' }}

    - name: Set password expiry policy
      ansible.builtin.lineinfile:
        path: /etc/login.defs
        regexp: "{{ item.regexp }}"
        line: "{{ item.line }}"
      loop:
        - { regexp: '^PASS_MAX_DAYS', line: 'PASS_MAX_DAYS 90' }
        - { regexp: '^PASS_MIN_DAYS', line: 'PASS_MIN_DAYS 7' }
        - { regexp: '^PASS_WARN_AGE', line: 'PASS_WARN_AGE 14' }
      when: ansible_memtotal_mb > 512

  handlers:
    - name: User setup complete
      ansible.builtin.debug:
```

```
        msg: "Handler triggered on {{ ansible_hostname }} — user provisioning complete"
```

■ append: true in the user module is critical — without it, Ansible replaces the user's group memberships with only wheel, removing them from any existing groups. Always use append: true when adding supplementary groups.

Module 3 Ansible Modules

dnf · service · file · copy · template · user · firewalld · command

How Modules Work

```
You write: Ansible does:

═══════════════════════════ ═══════════════════════════════════

- name: Install nginx 1. SSH to managed node

  ansible.builtin.dnf: 2. Generate Python script

    name: nginx 3. Copy and run it

    state: present 4. Capture result

                                5. Report ok/changed/failed

                                6. Clean up temp files
```

1. Package Management — dnf / package

```
  tasks:

    # Install a single package

    - name: Install nginx

      ansible.builtin.dnf:

        name: nginx

        state: present # present=installed, absent=removed

    # Install a list of packages

    - name: Install web stack

      ansible.builtin.dnf:

        name:

          - nginx

          - php-fpm

          - php-mysqlnd

          - git

        state: present

    # Update ALL packages

    - name: Full system update

      ansible.builtin.dnf:
```

```
        name: "*"
        state: latest

    # Install from a group
    - name: Install Development Tools group
      ansible.builtin.dnf:
        name: "@Development Tools"
        state: present

    # Cross-distro: use 'package' module
    - name: Install curl (works on any distro)
      ansible.builtin.package:
        name: curl
        state: present
```

2. Service Management — service / systemd

```
  tasks:

    # Start and enable a service
    - name: Start and enable nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true

    # Stop and disable
    - name: Stop and disable httpd
      ansible.builtin.service:
        name: httpd
        state: stopped
        enabled: false

    # Restart a service
    - name: Restart nginx
      ansible.builtin.service:
        name: nginx
```

```
      state: restarted

  # Reload config without full restart
  - name: Reload nginx
    ansible.builtin.service:
      name: nginx
      state: reloaded

  # Use systemd module for more control
  - name: Start nginx with systemd
    ansible.builtin.systemd:
      name: nginx
      state: started
      enabled: true
      daemon_reload: true # run systemctl daemon-reload first
```

State	What it does
started	Start if not running, do nothing if already running
stopped	Stop if running, do nothing if already stopped
restarted	Always restart
reloaded	Reload config — faster than restart

3. File and Directory Management — file

```
tasks:

  # Create a directory
  - name: Create app directory
    ansible.builtin.file:
      path: /opt/myapp
      state: directory
      owner: app
      group: app
      mode: '0755'

  # Create an empty file
  - name: Create log file
    ansible.builtin.file:
```

```
        path: /var/log/myapp.log
        state: touch
        owner: app
        group: app
        mode: '0644'

    # Remove a directory and all contents
    - name: Remove old app directory
      ansible.builtin.file:
        path: /opt/old-app
        state: absent

    # Create a symlink
    - name: Create symlink
      ansible.builtin.file:
        src: /opt/myapp-v2
        dest: /opt/myapp
        state: link

    # Change ownership recursively
    - name: Set ownership on web root
      ansible.builtin.file:
        path: /var/www/html
        owner: nginx
        group: nginx
        recurse: true
```

state	Effect
directory	Create directory (like mkdir -p)
file	Assert file exists, set attributes
touch	Create empty file if missing
absent	Remove file or directory
link	Create symbolic link
hard	Create hard link

4. Copy Files — copy / fetch

```
  tasks:
```

```yaml
  # Copy a local file to remote
  - name: Copy config file
    ansible.builtin.copy:
      src: files/nginx.conf # relative to playbook dir
      dest: /etc/nginx/nginx.conf
      owner: root
      group: root
      mode: '0644'
      backup: true # backup original as .timestamp

  # Copy inline content
  - name: Write a file with inline content
    ansible.builtin.copy:
      content: |
        # Managed by Ansible — do not edit manually
        server_name = web01
        port = 8080
      dest: /etc/myapp/config
      mode: '0640'

  # Fetch a file FROM remote TO control node
  - name: Fetch log file from remote
    ansible.builtin.fetch:
      src: /var/log/myapp.log
      dest: fetched_logs/
      flat: false
```

5. Edit Files — lineinfile / blockinfile

```yaml
tasks:

  # Ensure a line exists
  - name: Add DNS server
    ansible.builtin.lineinfile:
      path: /etc/resolv.conf
      line: "nameserver 8.8.8.8"
```

```
        state: present

    # Replace a line matching a pattern
    - name: Set SELINUX to permissive
      ansible.builtin.lineinfile:
        path: /etc/selinux/config
        regexp: '^SELINUX='
        line: 'SELINUX=permissive'

    # Insert a whole block
    - name: Add hosts block
      ansible.builtin.blockinfile:
        path: /etc/hosts
        block: |
          192.168.56.11 web01 web01.lab
          192.168.56.12 web02 web02.lab
          192.168.56.13 db01 db01.lab
        marker: "# {mark} ANSIBLE MANAGED BLOCK"
        state: present
```

6. Templates — template

Templates are files with Jinja2 variables. They live in a templates/ directory and use .j2 extension. Use copy for static files, template for files that vary per host.

```
{# templates/nginx.conf.j2 #}
# Managed by Ansible — do not edit manually
user {{ nginx_user | default('nginx') }};
worker_processes {{ ansible_processor_vcpus }};

events {
    worker_connections {{ nginx_worker_connections | default(1024) }};
}

http {
    server_tokens off;

    server {
        listen {{ http_port }};
```

```
        server_name {{ ansible_fqdn }};
        root {{ web_root }};

        {% if enable_https is defined and enable_https %}
        listen {{ https_port }} ssl;
        ssl_certificate /etc/ssl/certs/server.crt;
        {% endif %}
    }
}
```

```yaml
  tasks:
    - name: Deploy nginx config from template
      ansible.builtin.template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        owner: root
        group: root
        mode: '0644'
        validate: nginx -t -c %s # validate before deploying
      notify: Restart nginx
```

7. User and Group Management

```yaml
  tasks:

    # Create a group
    - name: Create developers group
      ansible.builtin.group:
        name: developers
        gid: 3000
        state: present

    # Create a user
    - name: Create application user
      ansible.builtin.user:
        name: appuser
        uid: 2500
```

```
      group: appuser # primary group
      groups: # supplementary groups
        - developers
        - docker
      append: true # don't remove existing groups
      shell: /bin/bash
      home: /opt/appuser
      create_home: true
      state: present

  # Create a system user (no home, no login)
  - name: Create service account
    ansible.builtin.user:
      name: myservice
      system: true # UID < 1000
      shell: /sbin/nologin
      create_home: false
      state: present

  # Remove a user and home directory
  - name: Remove old user
    ansible.builtin.user:
      name: olduser
      state: absent
      remove: true # delete home directory too
```

8. Firewall — firewalld

```
  tasks:

  # Open a service
  - name: Allow HTTP traffic
    ansible.builtin.firewalld:
      service: http
      state: enabled
      permanent: true
```

```
        immediate: true # apply now AND persist

    # Open a specific port
    - name: Open custom port 8080
      ansible.builtin.firewalld:
        port: 8080/tcp
        state: enabled
        permanent: true
        immediate: true

    # Add a rich rule
    - name: Allow SSH from specific subnet
      ansible.builtin.firewalld:
        rich_rule: 'rule family="ipv4" source address="192.168.56.0/24" service name="ssh" accept'
        state: enabled
        permanent: true
        immediate: true
```

> ■ Always set BOTH permanent: true AND immediate: true with firewalld. permanent alone survives reboots but doesn't apply now. immediate alone applies now but doesn't survive a reboot.

9. Command Execution — command / shell

```
  tasks:

    # command: no shell — safest
    - name: Initialize database
      ansible.builtin.command:
        cmd: postgresql-setup --initdb
        creates: /var/lib/pgsql/data/PG_VERSION # skip if file exists

    - name: Run migrations
      ansible.builtin.command:
        cmd: python3 manage.py migrate
        chdir: /opt/myapp # run from this dir

    # shell: full shell — use when you need pipes
    - name: Get active connections
```

```
      ansible.builtin.shell: ss -tnp | grep ESTABLISHED | wc -l
      register: conn_count
      changed_when: false

    # raw: no Python needed — use for bootstrapping only
    - name: Install Python on bare system
      ansible.builtin.raw: dnf install -y python3
      changed_when: true
```

Module	Use when
command	Simple commands, no shell features needed
shell	Need pipes, redirects, shell variables, globs
raw	Target has no Python — bootstrapping only
Native modules	ALWAYS prefer — idempotent, no guessing

10. Stat — Check Files Before Acting

```
  tasks:

    - name: Check if config exists
      ansible.builtin.stat:
        path: /etc/myapp/config
      register: config_file

    - name: Deploy default config if missing
      ansible.builtin.copy:
        src: files/default.conf
        dest: /etc/myapp/config
      when: not config_file.stat.exists

    # Useful stat fields:
    # stat.exists — True/False
    # stat.isdir — True if directory
    # stat.islink — True if symlink
    # stat.size — size in bytes
    # stat.mode — octal permissions e.g. '0644'
    # stat.owner — owner username
    # stat.checksum — sha1 checksum
```

11. Real-World — LAMP Stack Playbook

```yaml
# lamp_stack.yml
---
- name: Deploy LAMP stack
  hosts: webservers
  become: true

  vars:
    web_root: /var/www/html
    php_extensions:
      - php
      - php-fpm
      - php-mysqlnd
      - php-json
      - php-mbstring

  tasks:

    - name: Install Apache
      ansible.builtin.dnf:
        name: httpd
        state: present

    - name: Start and enable Apache
      ansible.builtin.service:
        name: httpd
        state: started
        enabled: true

    - name: Install PHP extensions
      ansible.builtin.dnf:
        name: "{{ php_extensions }}"
        state: present
      notify: Restart php-fpm

    - name: Open firewall for web traffic
      ansible.builtin.firewalld:
```

```
        service: "{{ item }}"
        state: enabled
        permanent: true
        immediate: true
      loop:
        - http
        - https

    - name: Set SELinux context on web root
      community.general.sefcontext:
        target: "{{ web_root }}(/.*)?
        setype: httpd_sys_content_t
        state: present

  handlers:
    - name: Restart httpd
      ansible.builtin.service:
        name: httpd
        state: restarted

    - name: Restart php-fpm
      ansible.builtin.service:
        name: php-fpm
        state: restarted
```

■ Key Takeaways

- dnf/package — install, remove, update packages — always prefer over shell: yum install
- service/systemd — started/stopped are idempotent; restarted always runs
- file — create dirs, files, symlinks, set permissions — covers mkdir, chmod, chown, ln
- copy — static files; template — files with variables; use validate: to test configs before deploying
- lineinfile — change one line; blockinfile — insert a whole block with markers
- user/group — always use append: true when adding supplementary groups
- firewalld — always set both permanent: true and immediate: true
- command for simple commands, shell only when you need pipes — prefer native modules always
- stat + register + when: — the pattern for conditional file operations

■ Module 3 Exercise — webserver_deploy.yml

Write a playbook webserver_deploy.yml that:

- Installs nginx, php-fpm, git using the dnf module
- Creates /var/www/myapp with correct nginx ownership

- Deploys an index.html using template module showing hostname and total RAM
- Uses lineinfile to ensure server_tokens off; is set in nginx.conf
- Opens ports 80 and 443 in the firewall using a loop
- Uses stat to check if /etc/nginx/ssl exists — if not, creates it
- Uses command with creates: to generate a self-signed SSL cert only if missing
- Starts and enables both nginx and php-fpm
- Has handlers for Restart nginx and Restart php-fpm

Module 3 Solution — webserver_deploy.yml

```
# templates/index.html.j2

<!DOCTYPE html>

<html>

<head><title>{{ ansible_hostname }} — Managed by Ansible</title></head>

<body style="background:#0d1117;color:#e6edf3;font-family:monospace;padding:40px">

   <h1 style="color:#58a6ff">Server Info</h1>

   <p>Hostname: <b>{{ ansible_hostname }}</b></p>

   <p>IP: <b>{{ ansible_default_ipv4.address }}</b></p>

   <p>OS: <b>{{ ansible_distribution }} {{ ansible_distribution_version }}</b></p>

   <p>RAM: <b>{{ ansible_memtotal_mb }} MB</b></p>

   <p>CPUs: <b>{{ ansible_processor_vcpus }}</b></p>

   <p>Deployed by Ansible: {{ ansible_date_time.date }}</p>

</body>

</html>
```

```
# webserver_deploy.yml

---

- name: Deploy and configure web servers

   hosts: webservers

   become: true


   vars:

     app_root: /var/www/myapp

     ssl_dir: /etc/nginx/ssl

     ssl_cert: /etc/nginx/ssl/server.crt

     ssl_key: /etc/nginx/ssl/server.key

     firewall_ports:

       - "80/tcp"

       - "443/tcp"
```

```yaml
tasks:

  - name: Install nginx, php-fpm, and git
    ansible.builtin.dnf:
      name:
        - nginx
        - php-fpm
        - git
      state: present

  - name: Create application web root
    ansible.builtin.file:
      path: "{{ app_root }}"
      state: directory
      owner: nginx
      group: nginx
      mode: '0755'

  - name: Deploy index.html from template
    ansible.builtin.template:
      src: templates/index.html.j2
      dest: "{{ app_root }}/index.html"
      owner: nginx
      group: nginx
      mode: '0644'
    notify: Restart nginx

  - name: Ensure server_tokens is off in nginx.conf
    ansible.builtin.lineinfile:
      path: /etc/nginx/nginx.conf
      regexp: '^\s*server_tokens'
      line: ' server_tokens off;'
      insertafter: 'http \{'
      backup: true
    notify: Restart nginx
```

```yaml
    - name: Open firewall ports for web traffic
      ansible.builtin.firewalld:
        port: "{{ item }}"
        state: enabled
        permanent: true
        immediate: true
      loop: "{{ firewall_ports }}"

    - name: Check if SSL directory exists
      ansible.builtin.stat:
        path: "{{ ssl_dir }}"
      register: ssl_dir_stat

    - name: Create SSL directory if missing
      ansible.builtin.file:
        path: "{{ ssl_dir }}"
        state: directory
        owner: root
        group: root
        mode: '0700'
      when: not ssl_dir_stat.stat.exists

    - name: Generate self-signed SSL certificate
      ansible.builtin.command:
        cmd: >
          openssl req -x509 -nodes -days 365
          -newkey rsa:2048
          -keyout {{ ssl_key }}
          -out {{ ssl_cert }}
          -subj "/CN={{ ansible_fqdn }}"
        creates: "{{ ssl_cert }}" # skip if cert already exists

    - name: Set permissions on SSL files
      ansible.builtin.file:
        path: "{{ item.path }}"
```

```yaml
        mode: "{{ item.mode }}"
      loop:
        - { path: "{{ ssl_cert }}", mode: '0644' }
        - { path: "{{ ssl_key }}", mode: '0600' }

    - name: Start and enable nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true

    - name: Start and enable php-fpm
      ansible.builtin.service:
        name: php-fpm
        state: started
        enabled: true

    - name: Verify nginx is responding
      ansible.builtin.uri:
        url: "http://{{ ansible_default_ipv4.address }}"
        status_code: 200
      register: nginx_check
      ignore_errors: true

  handlers:
    - name: Restart nginx
      ansible.builtin.service:
        name: nginx
        state: restarted

    - name: Restart php-fpm
      ansible.builtin.service:
        name: php-fpm
        state: restarted
```

■ command with creates: makes it idempotent — if the cert already exists, the openssl command is skipped entirely. This is the correct pattern for one-time setup tasks like certificate generation.

Module 4 Variables & Facts

ansible_facts · set_fact · hostvars · group_vars · Jinja2 filters

1. Ansible Facts — Auto-Discovered Variables

Every time Ansible connects to a host it runs the Gathering Facts task automatically. This collects hundreds of variables about the system.

```
# See ALL facts for a host
ansible web01 -m setup

# Filter facts by pattern
ansible web01 -m setup -a "filter=ansible_mem*"
ansible web01 -m setup -a "filter=ansible_distribution*"
```

Fact Variable	Example Value	Description
ansible_hostname	web01	Short hostname
ansible_fqdn	web01.lab.local	Fully qualified domain name
ansible_os_family	RedHat	OS family: RedHat / Debian
ansible_distribution	Rocky	Specific distro name
ansible_distribution_version	9.2	Full version
ansible_distribution_major_version	9	Major version only
ansible_kernel	5.14.0-284.el9	Kernel version
ansible_processor_vcpus	4	Number of vCPUs
ansible_memtotal_mb	16042	Total RAM in MB
ansible_default_ipv4.address	192.168.56.11	Primary IP address
ansible_default_ipv4.gateway	192.168.56.1	Default gateway
ansible_interfaces	['lo','eth0']	List of interfaces
ansible_mounts	[{mount:'/', ...}]	List of mount points
ansible_date_time.date	2024-01-15	Current date
ansible_date_time.iso8601	2024-01-15T14:30:00Z	ISO 8601 timestamp

Using Facts in Tasks

```
tasks:

  - name: Show system summary
    ansible.builtin.debug:
      msg: >
        {{ ansible_hostname }} —
        {{ ansible_distribution }} {{ ansible_distribution_version }} —
```

```
          {{ ansible_processor_vcpus }} CPUs —
          {{ ansible_memtotal_mb }}MB RAM

    # Different package manager based on OS family
    - name: Install htop (RedHat)
      ansible.builtin.dnf:
        name: htop
        state: present
      when: ansible_os_family == "RedHat"

    # Tune config based on available RAM
    - name: Set nginx worker connections based on RAM
      ansible.builtin.template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      vars:
        worker_connections: >-
          {{ 2048 if ansible_memtotal_mb >= 4096 else 1024 }}
```

2. Custom Facts — Define Your Own

```
# /etc/ansible/facts.d/myapp.fact — INI format
[application]
name = mywebapp
version = 2.4.1
env = production
port = 8080
```

```
# Deploy custom facts with Ansible
- name: Deploy custom facts file
  ansible.builtin.copy:
    content: |
      [application]
      name = mywebapp
      version = 2.4.1
      env = production
    dest: /etc/ansible/facts.d/myapp.fact
```

```
    mode: '0644'

- name: Reload facts after deploying
  ansible.builtin.setup:
    filter: ansible_local

- name: Use custom fact
  ansible.builtin.debug:
    msg: >-
      App: {{ ansible_local.myapp.application.name }}
      v{{ ansible_local.myapp.application.version }}"
```

3. set_fact — Compute Variables at Runtime

```
  tasks:

    # Compute a value from a fact
    - name: Calculate nginx worker count
      ansible.builtin.set_fact:
        nginx_workers: "{{ ansible_processor_vcpus * 2 }}"

    # Set fact based on condition
    - name: Set environment tier
      ansible.builtin.set_fact:
        server_tier: >-
          {{ 'high' if ansible_memtotal_mb >= 8192
             else 'standard' if ansible_memtotal_mb >= 2048
             else 'low' }}

    # Build a derived variable from registered output
    - name: Get listening ports
      ansible.builtin.shell: ss -tlnp | awk 'NR>1 {print $4}' | grep -oP ':\K\d+'
      register: listening_ports_raw
      changed_when: false

    - name: Set fact from registered output
      ansible.builtin.set_fact:
        listening_ports: "{{ listening_ports_raw.stdout_lines | map('int') | list }}"
```

```
    - name: Check if port 80 is open
      ansible.builtin.debug:
        msg: "Port 80 status: {{ 'OPEN' if 80 in listening_ports else 'CLOSED' }}"
```

4. Variable Scope and Precedence — Full Picture

```
HIGHEST ═══════════════════════════════════════════
  1. ansible-playbook -e "var=value" (extra vars — always wins)
  2. task vars
  3. block vars
  4. role and include vars
  5. set_fact / registered vars
  6. play vars (vars: in the play)
  7. play vars_files
  8. host_vars/hostname.yml
  9. group_vars/hostname
  10. group_vars/groupname.yml
  11. group_vars/all.yml
  12. inventory host variables
  13. inventory group variables
  14. role defaults (defaults/main.yml)
LOWEST ═══════════════════════════════════════════
```

5. hostvars — Access Another Host's Variables

```
  tasks:

    # Access the database server's IP from web servers
    - name: Configure app database connection
      ansible.builtin.template:
        src: templates/db_config.j2
        dest: /etc/myapp/database.conf

# templates/db_config.j2
# [database]
# host = {{ hostvars['db01'].ansible_default_ipv4.address }}
# port = {{ hostvars['db01'].db_port | default(5432) }}
```

```
  # Loop over all web servers from the DB play
  - name: Show all webserver IPs
    ansible.builtin.debug:
      msg: >-
        {% for host in groups['webservers'] %}
        {{ host }}: {{ hostvars[host].ansible_default_ipv4.address }}
        {% endfor %}"
```

6. Jinja2 Filters — Transform Variables

```
tasks:

  - name: String filter examples
    ansible.builtin.debug:
      msg:
        - "{{ 'hello world' | upper }}" # HELLO WORLD
        - "{{ 'NGINX' | lower }}" # nginx
        - "{{ ' spaces ' | trim }}" # spaces
        - "{{ 'web01' | replace('web', 'db') }}" # db01

  - name: Number filter examples
    ansible.builtin.debug:
      msg:
        - "{{ ansible_memtotal_mb | int }}"
        - "{{ ansible_memtotal_mb / 1024 | round(1) }}"

  - name: List filter examples
    ansible.builtin.debug:
      msg:
        - "{{ ['b','a','c'] | sort }}" # ['a','b','c']
        - "{{ ['a','b','a'] | unique }}" # ['a','b']
        - "{{ [1,2,3] | length }}" # 3
        - "{{ groups['webservers'] | first }}" # web01

  - name: Default values
    ansible.builtin.debug:
      msg:
```

```
          - "{{ undefined_var | default('fallback') }}"
          - "{{ http_port | default(80) }}"

    # Build list of IPs from a group
    - name: Get all webserver IPs
      ansible.builtin.set_fact:
        web_ips: >-
          {{ groups['webservers']
             | map('extract', hostvars, ['ansible_default_ipv4', 'address'])
             | list }}"
```

7. Disabling Fact Gathering

```
---
- name: Quick task — no facts needed
  hosts: all
  become: true
  gather_facts: false # skip Gathering Facts task

  tasks:
    - name: Restart a service
      ansible.builtin.service:
        name: crond
        state: restarted
```

8. Real-World — Dynamic Config Based on Facts

```
# dynamic_config.yml
---
- name: Configure servers dynamically using facts
  hosts: all
  become: true

  tasks:

    - name: Compute server profile
      ansible.builtin.set_fact:
        server_profile:
```

```
          hostname: "{{ ansible_hostname }}"
          ip: "{{ ansible_default_ipv4.address }}"
          ram_gb: "{{ (ansible_memtotal_mb / 1024) | round(1) }}"
          cpus: "{{ ansible_processor_vcpus }}"
          tier: >-
            {{ 'large' if ansible_memtotal_mb >= 8192 else
               'medium' if ansible_memtotal_mb >= 2048 else 'small' }}

    - name: Deploy tier-specific config
      ansible.builtin.copy:
        content: |
          # Auto-generated by Ansible — tier: {{ server_profile.tier }}
          {% if server_profile.tier == 'large' %}
          max_connections = 500
          {% elif server_profile.tier == 'medium' %}
          max_connections = 200
          {% else %}
          max_connections = 50
          {% endif %}
        dest: /etc/myapp/resources.conf

    - name: Update /etc/hosts with all managed nodes
      ansible.builtin.blockinfile:
        path: /etc/hosts
        block: |
          {% for host in groups['all'] %}
          {% if hostvars[host].ansible_default_ipv4 is defined %}
          {{ hostvars[host].ansible_default_ipv4.address }} {{
hostvars[host].ansible_hostname }}
          {% endif %}
          {% endfor %}
        marker: "# {mark} ANSIBLE MANAGED — inventory hosts"
```

■ Key Takeaways

- Facts are auto-collected before every play — ansible_hostname, ansible_memtotal_mb, ansible_distribution are the ones you'll use daily

- ansible web01 -m setup -a 'filter=ansible_mem*' — always inspect facts this way before writing conditionals
- Custom facts live in /etc/ansible/facts.d/*.fact — accessible as ansible_local
- set_fact computes variables mid-play — use it to derive values from facts or registered output
- hostvars['hostname'].variable — read any other host's variables from any task
- group_vars/all.yml -> group_vars/groupname.yml -> host_vars/hostname.yml is the right layering
- Filters transform variables inline: | default(), | int, | round(), | upper, | join(), | unique
- gather_facts: false speeds up plays that don't use facts

■ Module 4 Exercise — facts_report.yml

Create a playbook facts_report.yml that:

- Uses set_fact to compute: ram_gb (total RAM rounded to 1 decimal), disk_free_gb (free space on / mount), server_class (large if RAM >= 8GB, medium if >= 2GB, else small)
- Creates /opt/ansible-managed/server_report.txt on each host using a template with hostname, IP, OS, kernel, CPUs, RAM, disk free, server class
- Uses hostvars and a Jinja2 loop to build a mini hosts table of ALL managed hosts
- Uses debug with a filter to print server_class | upper for each host
- Only installs htop on large and medium class servers using when:
- Fetches the completed report back to control node into ./reports/

Module 4 Solution — facts_report.yml

```
# templates/server_report.j2

=====================================================

   SERVER REPORT — {{ ansible_hostname }}

   Generated : {{ ansible_date_time.date }} {{ ansible_date_time.time }}

=====================================================


   SYSTEM IDENTITY

   Hostname : {{ ansible_hostname }}

   FQDN : {{ ansible_fqdn }}

   IP Address : {{ ansible_default_ipv4.address }}

   Server Class : {{ server_class | upper }}


   HARDWARE

   CPUs : {{ ansible_processor_vcpus }}

   Total RAM : {{ ram_gb }} GB

   Disk Free : {{ disk_free_gb }} GB (on /)


   OPERATING SYSTEM

   Distribution : {{ ansible_distribution }} {{ ansible_distribution_version }}

   Kernel : {{ ansible_kernel }}


   ALL MANAGED HOSTS IN INVENTORY
```

```
   {{ '%-20s' | format('HOSTNAME') }} {{ 'IP ADDRESS' }}

   {{ '-' * 42 }}

{% for host in groups['all'] | sort %}

{% if hostvars[host].ansible_default_ipv4 is defined %}

   {{ '%-20s' | format(hostvars[host].ansible_hostname) }} {{
hostvars[host].ansible_default_ipv4.address }}

{% endif %}

{% endfor %}

==================================================
```

```
# facts_report.yml

---

- name: Collect facts and generate server reports

   hosts: all

   become: true

   vars:

     report_dir: /opt/ansible-managed

     report_file: /opt/ansible-managed/server_report.txt

     fetch_dest: ./reports/

   tasks:

     - name: Compute RAM in GB

       ansible.builtin.set_fact:

         ram_gb: "{{ (ansible_memtotal_mb / 1024) | round(1) }}"

     - name: Compute free disk space on root partition

       ansible.builtin.set_fact:

         disk_free_gb: >-

           {{

             (ansible_mounts

              | selectattr('mount', 'equalto', '/')

              | map(attribute='size_available')

              | first

              / 1024 / 1024 / 1024)

             | round(1)
```

```yaml
      }}

  - name: Compute server class from RAM
    ansible.builtin.set_fact:
      server_class: >-
        {{ 'large' if ansible_memtotal_mb >= 8192 else
           'medium' if ansible_memtotal_mb >= 2048 else
           'small' }}

  - name: Display server class
    ansible.builtin.debug:
      msg: >-
        {{ ansible_hostname }} —
        {{ ram_gb }}GB RAM —
        class: {{ server_class | upper }}

  - name: Ensure managed directory exists
    ansible.builtin.file:
      path: "{{ report_dir }}"
      state: directory
      owner: root
      group: root
      mode: '0755'

  - name: Generate server report from template
    ansible.builtin.template:
      src: templates/server_report.j2
      dest: "{{ report_file }}"
      owner: root
      group: root
      mode: '0644'

  - name: Install htop on large and medium servers
    ansible.builtin.dnf:
      name: htop
      state: present
```

```
      when: server_class in ['large', 'medium']

    - name: Fetch server report to control node
      ansible.builtin.fetch:
        src: "{{ report_file }}"
        dest: "{{ fetch_dest }}"
        flat: false
```

> ■ selectattr + map + first is the standard pattern to extract a single field from a list of dicts. ansible_mounts is a list — selectattr filters to the root mount, map extracts size_available, first gets the value.

Module 5 Roles

Role structure · defaults vs vars · Galaxy · site.yml

Why Roles Exist

```
Without roles — one big playbook: With roles — clean separation:

■■■
```

```
ansible-galaxy role init roles/nginx

ansible-galaxy role init roles/postgresql

ansible-galaxy role init roles/hardening
```

2. Building the nginx Role — Complete Example

```yaml
# roles/nginx/defaults/main.yml

# LOW precedence — users override these in group_vars or playbooks

---

nginx_user: nginx

nginx_port: 80

nginx_ssl_port: 443

nginx_worker_processes: auto

nginx_worker_connections: 1024

nginx_server_name: "{{ ansible_fqdn }}"

nginx_web_root: /var/www/html

nginx_enable_ssl: false

nginx_ssl_cert: /etc/nginx/ssl/server.crt

nginx_ssl_key: /etc/nginx/ssl/server.key

nginx_server_tokens: off

nginx_extra_packages: []
```

```yaml
# roles/nginx/vars/main.yml

# HIGH precedence — implementation details, not meant to be overridden

---

nginx_config_path: /etc/nginx/nginx.conf

nginx_vhost_dir: /etc/nginx/conf.d

nginx_log_dir: /var/log/nginx

nginx_ssl_dir: /etc/nginx/ssl
```

```yaml
# roles/nginx/tasks/main.yml

---

- name: Import installation tasks

  ansible.builtin.import_tasks: install.yml


- name: Import configuration tasks

  ansible.builtin.import_tasks: configure.yml
```

```yaml
- name: Import SSL tasks
  ansible.builtin.import_tasks: ssl.yml
  when: nginx_enable_ssl | bool

- name: Import firewall tasks
  ansible.builtin.import_tasks: firewall.yml

- name: Import service tasks
  ansible.builtin.import_tasks: service.yml
```

```yaml
# roles/nginx/tasks/install.yml
---
- name: Install nginx
  ansible.builtin.dnf:
    name: "{{ ['nginx'] + nginx_extra_packages }}"
    state: present

- name: Create web root directory
  ansible.builtin.file:
    path: "{{ nginx_web_root }}"
    state: directory
    owner: "{{ nginx_user }}"
    group: "{{ nginx_user }}"
    mode: '0755'
```

```yaml
# roles/nginx/tasks/configure.yml
---
- name: Deploy nginx main config
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: "{{ nginx_config_path }}"
    mode: '0644'
    validate: nginx -t -c %s
  notify: Restart nginx

- name: Deploy default virtual host
  ansible.builtin.template:
```

```
    src: vhost.conf.j2
    dest: "{{ nginx_vhost_dir }}/default.conf"
    mode: '0644'
  notify: Reload nginx
```

```
# roles/nginx/handlers/main.yml
---
- name: Restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted

- name: Reload nginx
  ansible.builtin.service:
    name: nginx
    state: reloaded
```

```
# roles/nginx/meta/main.yml
---
galaxy_info:
  role_name: nginx
  author: yourname
  description: Install and configure nginx web server
  license: MIT
  min_ansible_version: "2.14"
  platforms:
    - name: EL
      versions: ["8", "9"]
  galaxy_tags: [nginx, web, rhel]

dependencies: []
```

3. Using Roles in a Playbook

```
# site.yml
---
- name: Configure web servers
```

```
  hosts: webservers
  become: true

  roles:
    - nginx # uses all defaults

- name: Configure database servers
  hosts: databases
  become: true

  roles:
    - postgresql
```

```
# Override role defaults per play
- name: Configure web servers
  hosts: webservers
  become: true

  roles:
    - role: nginx
      vars:
        nginx_port: 8080
        nginx_enable_ssl: true

# Or override for the whole group in group_vars/webservers.yml:
# nginx_port: 80
# nginx_enable_ssl: true
```

4. import_role vs include_role

	import_role	include_role
When processed	At parse time — static	At runtime — dynamic
Works with when:	Applies to all role tasks	Applies to the include itself
Works with loop:	No	Yes
Use when	Default choice	Need dynamic role selection

5. Ansible Galaxy — Using Community Roles

```
# Search for roles
ansible-galaxy search nginx
```

```
# Install a role from Galaxy
ansible-galaxy role install geerlingguy.nginx
ansible-galaxy role install geerlingguy.postgresql

# Install from requirements file
ansible-galaxy role install -r requirements.yml

# requirements.yml — pin all dependencies
---
roles:
  - name: geerlingguy.nginx
    version: "3.2.0"
  - name: geerlingguy.postgresql
    version: "3.4.0"

collections:
  - name: ansible.posix
    version: ">=1.4.0"
  - name: community.general
    version: ">=7.0.0"
  - name: community.postgresql
    version: ">=2.3.0"
```

6. Real-World site.yml with Multiple Roles

```
# site.yml — the master playbook
---

# Apply hardening to everything
- name: Harden all servers
  hosts: all
  become: true
  roles:
    - role: hardening
      vars:
        hardening_disable_root_login: true
        hardening_max_auth_tries: 3
```

```yaml
# Configure web servers
- name: Configure web servers
  hosts: webservers
  become: true
  roles:
    - role: nginx
      vars:
        nginx_enable_ssl: true
        nginx_worker_connections: >-
          {{ 2048 if ansible_memtotal_mb >= 4096 else 1024 }}

# Configure database servers
- name: Configure database servers
  hosts: databases
  become: true
  roles:
    - role: postgresql
      vars:
        postgresql_max_connections: >-
          {{ 200 if ansible_memtotal_mb >= 8192 else 100 }}
        postgresql_databases:
          - name: webapp_db
          - name: analytics_db
        postgresql_users:
          - name: webapp
            password: "{{ vault_webapp_db_password }}"
            db: webapp_db
```

```bash
# Full deploy
ansible-playbook site.yml

# Only run hardening role using tags
ansible-playbook site.yml --tags hardening

# Only web servers
```

```
ansible-playbook site.yml --limit webservers

# Dry run everything
ansible-playbook site.yml --check
```

■ Key Takeaways

- Roles split playbooks into reusable, shareable units — tasks/, handlers/, templates/, files/, vars/, defaults/, meta/
- defaults/main.yml — low precedence, meant to be overridden by users
- vars/main.yml — high precedence, implementation internals not meant to be overridden
- ansible-galaxy role init roles/rolename generates the skeleton instantly
- import_role is static (parse time), include_role is dynamic (run time) — default to import_role
- requirements.yml pins all Galaxy dependencies — always use it in real projects
- Split long tasks/main.yml into subtask files and import_tasks: them
- no_log: true hides sensitive values like passwords from playbook output

■ Module 5 Exercise — baseline Role

Create a role called baseline that:

- Has defaults/main.yml with: baseline_packages, baseline_timezone (UTC), baseline_managed_dir (/opt/ansible-managed), baseline_motd_enabled (true)
- tasks/main.yml imports 4 sub-task files: packages.yml, system.yml, motd.yml, report.yml
- packages.yml — installs baseline_packages
- system.yml — sets timezone, creates baseline_managed_dir, sets hostname
- motd.yml — only runs when baseline_motd_enabled is true — deploys /etc/motd from a template
- report.yml — writes a completion report with timestamp and all key facts
- Handler: Restart chronyd triggered from the timezone task
- site.yml applies baseline to all hosts, then nginx to webservers — overriding baseline_timezone to America/New_York for webservers

Module 5 Solution — baseline Role

```
roles/
■■■ baseline/
    ■■■ defaults/main.yml
    ■■■ tasks/
    ■ ■■■ main.yml
    ■ ■■■ packages.yml
    ■ ■■■ system.yml
    ■ ■■■ motd.yml
    ■ ■■■ report.yml
    ■■■ templates/
    ■ ■■■ motd.j2
    ■ ■■■ baseline_report.j2
    ■■■ handlers/main.yml
```

```
     ■■■ meta/main.yml
     ■■■ README.md
```

```yaml
# roles/baseline/defaults/main.yml
---
baseline_packages:
   - vim
   - curl
   - wget
   - net-tools
   - bash-completion
   - htop

baseline_timezone: UTC
baseline_managed_dir: /opt/ansible-managed
baseline_motd_enabled: true
```

```yaml
# roles/baseline/tasks/main.yml
---
- name: Import package tasks
   ansible.builtin.import_tasks: packages.yml

- name: Import system configuration tasks
   ansible.builtin.import_tasks: system.yml

- name: Import MOTD tasks
   ansible.builtin.import_tasks: motd.yml

- name: Import report tasks
   ansible.builtin.import_tasks: report.yml
```

```yaml
# roles/baseline/tasks/packages.yml
---
- name: Install baseline packages
   ansible.builtin.dnf:
     name: "{{ baseline_packages }}"
     state: present
     update_cache: true
```

```yaml
# roles/baseline/tasks/system.yml
---
- name: Set system timezone
  community.general.timezone:
    name: "{{ baseline_timezone }}"
  notify: Restart chronyd

- name: Create managed directory
  ansible.builtin.file:
    path: "{{ baseline_managed_dir }}"
    state: directory
    owner: root
    group: root
    mode: '0755'

- name: Set system hostname
  ansible.builtin.hostname:
    name: "{{ ansible_hostname }}"
    use: systemd

# roles/baseline/tasks/motd.yml
---
- name: Deploy MOTD from template
  ansible.builtin.template:
    src: motd.j2
    dest: /etc/motd
    owner: root
    group: root
    mode: '0644'
  when: baseline_motd_enabled | bool
```

```
# roles/baseline/templates/motd.j2
■■■
```

```
  Hostname : {{ ansible_hostname }}

  FQDN : {{ ansible_fqdn }}

  IP Address : {{ ansible_default_ipv4.address }}

  OS : {{ ansible_distribution }} {{ ansible_distribution_version }}

  RAM : {{ (ansible_memtotal_mb / 1024) | round(1) }} GB

  Timezone : {{ baseline_timezone }}


  All activity on this system is monitored and logged.
```

```
# roles/baseline/handlers/main.yml

---

- name: Restart chronyd

  ansible.builtin.service:

    name: chronyd

    state: restarted
```

```
# site.yml — Master Playbook

---


# Play 1: Baseline for ALL hosts

- name: Apply baseline role to all managed hosts

  hosts: all

  become: true

  roles:

    - role: baseline # uses all defaults from defaults/main.yml


# Play 2: Webservers — baseline override + nginx

- name: Configure web servers

  hosts: webservers

  become: true

  roles:

    - role: baseline

      vars:

        baseline_timezone: America/New_York # override for webservers


    - role: nginx

      vars:
```

```
        nginx_enable_ssl: true
```

■ defaults/main.yml is LOW precedence — designed to be overridden. vars/main.yml is HIGH precedence — implementation internals. Putting baseline_timezone in defaults/ is what makes the America/New_York override work.

Module 6 Jinja2 Templates In Depth

Loops · Conditionals · Filters · Macros · Production config patterns

1. Jinja2 Syntax Fundamentals

```
{# Three delimiter types: #}

{{ variable }} {# Output a variable or expression #}

{% statement %} {# Control flow — if, for, set, etc #}

{# comment #} {# Comment — stripped from output #}


{# Whitespace control — add - to trim surrounding whitespace #}

{{- variable -}} {# strip whitespace before AND after #}

{%- if condition -%} {# strip whitespace around block tag #}
```

2. Variables and Expressions

```
{# Basic variable output #}

Hostname: {{ ansible_hostname }}

RAM: {{ ansible_memtotal_mb }} MB


{# Arithmetic #}

RAM in GB: {{ (ansible_memtotal_mb / 1024) | round(2) }}

Double CPUs: {{ ansible_processor_vcpus * 2 }}


{# String operations #}

Upper: {{ ansible_hostname | upper }}

Replace: {{ ansible_hostname | replace('web', 'srv') }}


{# Ternary expression — inline if/else #}

Size class: {{ 'large' if ansible_memtotal_mb >= 8192 else 'small' }}


{# Default values #}

Port: {{ custom_port | default(80) }}

User: {{ app_user | default('nginx') }}

Debug: {{ debug_mode | default(false) | bool }}
```

3. Conditionals in Templates

```
{# templates/nginx_vhost.conf.j2 #}
```

```
server {
    listen {{ nginx_port | default(80) }};
    server_name {{ ansible_fqdn }};

    {# Simple if/else #}
    {% if nginx_enable_ssl | bool %}
    listen {{ nginx_ssl_port | default(443) }} ssl;
    ssl_certificate {{ nginx_ssl_cert }};
    ssl_certificate_key {{ nginx_ssl_key }};
    ssl_protocols TLSv1.2 TLSv1.3;

        {% if nginx_hsts_enabled | default(false) | bool %}
    add_header Strict-Transport-Security "max-age=31536000" always;
        {% endif %}
    {% endif %}

    {# if/elif/else #}
    {% if env == 'production' %}
    error_log /var/log/nginx/error.log warn;
    {% elif env == 'staging' %}
    error_log /var/log/nginx/error.log info;
    {% else %}
    error_log /var/log/nginx/error.log debug;
    {% endif %}

    {# Check if variable is defined #}
    {% if nginx_client_max_body_size is defined %}
    client_max_body_size {{ nginx_client_max_body_size }};
    {% endif %}
}
```

4. Loops in Templates

```
{# templates/haproxy.cfg.j2 #}

frontend web_frontend
    bind *:80
```

```
    default_backend web_servers

backend web_servers
    balance roundrobin
{% for host in groups['webservers'] %}
    server {{ hostvars[host].ansible_hostname }} \
           {{ hostvars[host].ansible_default_ipv4.address }}:80 \
           check
{% endfor %}

{# Loop with index — loop.index starts at 1 #}
{% for server in backend_servers %}
    server backend{{ loop.index }} {{ server.ip }}:{{ server.port }} check
    {%- if loop.last %} # last server{%- endif %}
{% endfor %}

{# Loop over a dict #}
{% for key, value in nginx_headers.items() %}
    add_header {{ key }} "{{ value }}";
{% endfor %}
```

Variable	Value
loop.index	Current iteration (1-based)
loop.index0	Current iteration (0-based)
loop.first	True on first iteration
loop.last	True on last iteration
loop.length	Total number of items
loop.revindex	Iterations from end (1-based)

5. Jinja2 Filters — Complete Reference

```
{# String filters #}
{{ 'hello' | upper }} {# HELLO #}
{{ 'HELLO' | lower }} {# hello #}
{{ 'hello world' | title }} {# Hello World #}
{{ ' spaces ' | trim }} {# spaces #}
{{ 'hello world' | replace('o','0') }} {# hell0 w0rld #}
{{ 'hello' | truncate(8) }} {# hello... #}
```

```
{{ ['a','b','c'] | join(', ') }} {# a, b, c #}
{{ ansible_hostname | regex_replace('^web', 'srv') }}

{# Number filters #}
{{ '42' | int }} {# 42 #}
{{ 3.14159 | round(2) }} {# 3.14 #}
{{ 1073741824 | human_readable }} {# 1.0 GB #}

{# List filters #}
{{ ['b','a','c'] | sort }} {# ['a','b','c'] #}
{{ ['a','b','a','c'] | unique }} {# ['a','b','c'] #}
{{ [3,1,2] | min }} {# 1 #}
{{ [3,1,2] | max }} {# 3 #}
{{ [1,2,3] | sum }} {# 6 #}
{{ [1,2,3] | length }} {# 3 #}
{{ [1,2,3,4,5] | select('odd') | list }} {# [1,3,5] #}
{{ ['a','b','c'] | first }} {# a #}
{{ ['a','b','c'] | last }} {# c #}

{# Dict filters #}
{{ mydict | dict2items }} {# convert to list of {key,value} dicts #}
{{ mylist | items2dict }} {# convert back to dict #}
{{ mydict | combine(other_dict) }} {# merge two dicts #}

{# Test filters #}
{{ myvar is defined }}
{{ myvar is undefined }}
{{ myvar | bool }}
{{ myvar | ternary('yes','no') }} {# 'yes' if truthy, 'no' if falsy #}

{# Hash / encoding filters #}
{{ 'password123' | password_hash('sha512') }}
{{ 'hello' | b64encode }} {# aGVsbG8= #}
{{ 'aGVsbG8=' | b64decode }} {# hello #}
{{ myvar | to_json }}
{{ myvar | to_yaml }}
```

6. Jinja2 Macros — Reusable Template Functions

```
{# templates/macros/common.j2 #}

{% macro server_block(name, ip, port=80, weight=1) %}
     server {{ name }} {{ ip }}:{{ port }} weight={{ weight }} check;
{% endmacro %}

{% macro section_header(title, char='=') %}
{{ char * 50 }}
   {{ title }}
{{ char * 50 }}
{% endmacro %}

{# Using macros in another template #}
{% import 'macros/common.j2' as common %}

{{ common.section_header('HAProxy Configuration') }}

backend web_pool
     balance roundrobin
{% for host in groups['webservers'] %}
     {{ common.server_block(
          hostvars[host].ansible_hostname,
          hostvars[host].ansible_default_ipv4.address
        ) }}
{% endfor %}
```

7. Real-World Templates — Production Configs

```
{# templates/sshd_config.j2 #}
# Managed by Ansible — do not edit manually
# Last applied: {{ ansible_date_time.iso8601 }}

Port {{ ssh_port | default(22) }}
AddressFamily {{ ssh_address_family | default('inet') }}

# Authentication
PermitRootLogin {{ ssh_permit_root_login | default('no') }}
PasswordAuthentication {{ ssh_password_auth | default('no') }}
```

```
PubkeyAuthentication yes

MaxAuthTries {{ ssh_max_auth_tries | default(3) }}

{% if ssh_allowed_users is defined and ssh_allowed_users | length > 0 %}

AllowUsers {{ ssh_allowed_users | join(' ') }}

{% endif %}

# Session

ClientAliveInterval {{ ssh_alive_interval | default(300) }}

ClientAliveCountMax {{ ssh_alive_count | default(2) }}

# Features

X11Forwarding {{ ssh_x11_forwarding | default('no') }}

Banner {{ ssh_banner | default('/etc/issue.net') }}

Subsystem sftp /usr/lib/openssh/sftp-server
```

```
{# templates/limits.conf.j2 #}

# /etc/security/limits.conf

# Managed by Ansible

* soft nofile {{ limits_nofile_soft | default(65536) }}

* hard nofile {{ limits_nofile_hard | default(131072) }}

* soft nproc {{ limits_nproc_soft | default(65536) }}

* hard nproc {{ limits_nproc_hard | default(131072) }}

{% if 'databases' in group_names %}

# Database server tuning

postgres soft nofile 65536

postgres hard nofile 65536

{% endif %}

{% if 'webservers' in group_names %}

# Web server tuning

nginx soft nofile 65536

nginx hard nofile 65536

{% endif %}
```

■ Key Takeaways

- {{ }} outputs values, {% %} is control flow, {# #} is comments
- Use {{- -}} with dashes to strip surrounding whitespace when formatting matters
- {% if %} / {% elif %} / {% else %} / {% endif %} — full conditionals in templates
- {% for item in list %} / {% endfor %} — loops with loop.index, loop.first, loop.last
- | default('value') is essential — always use it for optional variables in templates
- | bool converts string 'true'/'false'/'yes'/'no' to actual booleans — always use on boolean vars
- Macros are template functions — define once with {% macro %}, import with {% import %}
- group_names inside templates gives current host's group memberships — use for group-specific config blocks
- hostvars inside templates lets you build configs that reference other hosts — HAProxy, firewall rules etc.
- Always use validate: when deploying critical configs like sshd_config and nginx.conf

■ Module 6 Exercise

Create a template-driven configuration system:

- templates/app_config.j2 — app config with db_host from hostvars, SSL section with {% if %}, allowed IPs with {% for %}, webserver peers loop
- templates/hosts_file.j2 — generates /etc/hosts looping over groups['all'] with loop.index comments
- templates/cron_jobs.j2 — generates cron entries from a cron_jobs list, with | default('*') for optional fields
- deploy_configs.yml — deploys all three templates with proper validation

Module 6 Solution — Template-Driven Configuration

```
{# templates/app_config.j2 #}

# Generated : {{ ansible_date_time.date }} {{ ansible_date_time.time }}

# Host : {{ ansible_hostname }}


[general]

hostname = {{ ansible_hostname }}

environment = {{ app_env | default('production') }}

port = {{ app_port | default(8080) }}

log_level = {{ app_log_level | default('info') }}

max_connections = {{ app_max_connections | default(100) }}

worker_threads = {{ ansible_processor_vcpus * 2 }}


[database]

{% if 'db01' in hostvars and hostvars['db01'].ansible_default_ipv4 is defined %}

host = {{ hostvars['db01'].ansible_default_ipv4.address }}

{% else %}

host = localhost

{% endif %}

port = {{ app_db_port | default(5432) }}

name = {{ app_db_name | default('appdb') }}
```

```
{% if app_ssl_enabled | default(false) | bool %}
[ssl]
enabled = true
cert_file = {{ app_ssl_cert | default('/etc/ssl/certs/app.crt') }}
key_file = {{ app_ssl_key | default('/etc/ssl/private/app.key') }}
{% else %}
[ssl]
enabled = false
{% endif %}

[network]
{% for ip in app_allowed_ips | default([]) %}
allowed_ip_{{ loop.index }} = {{ ip }}
{% endfor %}

[peers]
peer_count = {{ groups['webservers'] | length }}
{% for host in groups['webservers'] | sort %}
{% if hostvars[host].ansible_default_ipv4 is defined %}
peer_{{ loop.index }}_name = {{ hostvars[host].ansible_hostname }}
peer_{{ loop.index }}_ip = {{ hostvars[host].ansible_default_ipv4.address }}
{% endif %}
{% endfor %}
```

```
{# templates/hosts_file.j2 #}
# /etc/hosts — Managed by Ansible
# Generated: {{ ansible_date_time.date }}

127.0.0.1 localhost localhost.localdomain
::1 localhost localhost.localdomain

# Managed Hosts ({{ groups['all'] | length }} total)
{% set ns = namespace(count=0) %}
{% for host in groups['all'] | sort %}
{% if hostvars[host].ansible_default_ipv4 is defined %}
{% set ns.count = ns.count + 1 %}
```

```
{{ '%-18s' | format(hostvars[host].ansible_default_ipv4.address) }} {{
hostvars[host].ansible_hostname }} # host {{ ns.count }}

{% endif %}

{% endfor %}
```

```
{# templates/cron_jobs.j2 #}

# /etc/cron.d/ansible_jobs

# Host: {{ ansible_hostname }} — Generated: {{ ansible_date_time.date }}


SHELL=/bin/bash

PATH=/sbin:/bin:/usr/sbin:/usr/bin

MAILTO=root


{% for job in cron_jobs %}

# Job {{ loop.index }}: {{ job.name }}

{{ job.minute | default('*') }} {{ job.hour | default('*') }} {{ job.day | default('*') }}
{{ job.month | default('*') }} {{ job.weekday | default('*') }} root {{ job.job }}


{% endfor %}


{% if ansible_memtotal_mb < 2048 %}

# Low-memory host ({{ ansible_memtotal_mb }}MB) — extra cleanup

*/30 * * * * root sync && echo 3 > /proc/sys/vm/drop_caches

0 4 * * * root find /tmp -type f -mtime +1 -delete

{% endif %}
```

```
# deploy_configs.yml

---

- name: Deploy template-driven configurations

  hosts: all

  become: true


  tasks:


    - name: Create app config directory

      ansible.builtin.file:

        path: /etc/myapp

        state: directory

        mode: '0755'
```

```yaml
- name: Deploy application configuration
  ansible.builtin.template:
    src: templates/app_config.j2
    dest: /etc/myapp/app.conf
    mode: '0640'
    backup: true

- name: Deploy /etc/hosts from template
  ansible.builtin.template:
    src: templates/hosts_file.j2
    dest: /etc/hosts
    mode: '0644'
    backup: true

- name: Deploy cron jobs file
  ansible.builtin.template:
    src: templates/cron_jobs.j2
    dest: /etc/cron.d/ansible_jobs
    mode: '0644'
    validate: crontab -u root %s
```

■ namespace() is required when you need a counter inside a Jinja2 loop. Regular variables set inside loops are invisible outside them — namespace() creates an object whose attributes persist across loop iterations.

Module 7 Advanced Topics

Vault · Tags · block/rescue/always · delegate_to · serial · async

1. Ansible Vault — Encrypting Secrets

Never store passwords, API keys, or certificates in plain text in your repo. Vault encrypts them with AES256.

```
# Create an encrypted file
ansible-vault create group_vars/all/vault.yml

# Encrypt an existing file
ansible-vault encrypt group_vars/production/vault.yml

# View encrypted file
ansible-vault view group_vars/all/vault.yml

# Edit encrypted file
ansible-vault edit group_vars/all/vault.yml

# Decrypt a file permanently
ansible-vault decrypt group_vars/all/vault.yml

# Change the vault password
ansible-vault rekey group_vars/all/vault.yml

# Encrypt a single string inline
ansible-vault encrypt_string 'MySecretPassword' --name 'vault_db_password'
```

```
# Vault file structure — best practice
group_vars/
■■■ all/
■ ■■■ vars.yml ← plain variables
■ ■■■ vault.yml ← encrypted secrets (prefix with vault_)
■■■ webservers/
■ ■■■ vars.yml
■ ■■■ vault.yml
■■■ databases/
    ■■■ vars.yml
    ■■■ vault.yml
```

```yaml
# group_vars/all/vault.yml — ENCRYPTED
---
vault_db_password: "SuperSecretDBPass123!"
vault_app_secret_key: "xK9mN2pQ8rT5vW3yA6bD"
vault_admin_password: "AdminPass456!"

# group_vars/all/vars.yml — plain text references vault vars
---
db_password: "{{ vault_db_password }}"
app_secret_key: "{{ vault_app_secret_key }}"
admin_password: "{{ vault_admin_password }}"
```

```bash
# Run playbook with vault

# Prompt for password interactively
ansible-playbook site.yml --ask-vault-pass

# Use a password file (for CI/CD)
echo "MyVaultPassword" > ~/.vault_pass
chmod 600 ~/.vault_pass
ansible-playbook site.yml --vault-password-file ~/.vault_pass

# Set in ansible.cfg so you never forget:
# [defaults]
# vault_password_file = ~/.vault_pass
```

2. Tags — Run Only What You Need

```yaml
# site.yml with tags
---
- name: Configure all servers
  hosts: all
  become: true

  tasks:

    - name: Install base packages
      ansible.builtin.dnf:
        name: "{{ base_packages }}"
```

```yaml
        state: present
      tags:
        - packages
        - baseline

    - name: Set timezone
      community.general.timezone:
        name: "{{ timezone }}"
      tags:
        - system
        - baseline
        - timezone

    - name: Deploy nginx config
      ansible.builtin.template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      tags:
        - nginx
        - config
        - never # only runs when explicitly called

    - name: Restart all services
      ansible.builtin.service:
        name: "{{ item }}"
        state: restarted
      loop: "{{ managed_services }}"
      tags:
        - restart
        - never # won't run in normal execution
```

```bash
# Running tagged tasks

# Run ONLY tasks tagged 'packages'
ansible-playbook site.yml --tags packages

# Run ONLY 'baseline' tag (installs + timezone)
```

```
ansible-playbook site.yml --tags baseline

# Run multiple tags
ansible-playbook site.yml --tags "packages,system"

# Skip specific tags
ansible-playbook site.yml --skip-tags nginx

# Run tasks tagged 'never' (opt-in tasks)
ansible-playbook site.yml --tags restart

# List all tags in a playbook
ansible-playbook site.yml --list-tags
```

Special Tag	Behavior
always	Runs EVERY time — even with --tags other_tag
never	Only runs when explicitly called with --tags never
tagged	Run all tagged tasks
untagged	Run all untagged tasks

```
# Tag an entire role
- name: Configure web servers
  hosts: webservers
  roles:
    - role: nginx
      tags: nginx # ALL tasks in the role get this tag

    - role: hardening
      tags:
        - hardening
        - security
```

3. Error Handling — block / rescue / always

block/rescue/always is the Ansible equivalent of try/except/finally.

```
  tasks:

    - name: Deploy application with rollback
      block:

        - name: Stop application
```

```yaml
      ansible.builtin.service:
        name: myapp
        state: stopped

    - name: Deploy new version
      ansible.builtin.copy:
        src: files/myapp-v2.tar.gz
        dest: /opt/myapp/myapp.tar.gz

    - name: Start application
      ansible.builtin.service:
        name: myapp
        state: started

  rescue:
    # Runs if ANY task in block fails
    - name: Log deployment failure
      ansible.builtin.debug:
        msg: "Deployment failed on {{ ansible_hostname }} — initiating rollback"

    - name: Restore previous version
      ansible.builtin.shell: |
        cp -r /opt/myapp-backup/. /opt/myapp/

    - name: Restart with previous version
      ansible.builtin.service:
        name: myapp
        state: restarted

  always:
    # Runs whether block succeeded or failed
    - name: Verify application is running
      ansible.builtin.uri:
        url: "http://localhost:8080/health"
        status_code: 200
      ignore_errors: true
```

```
      - name: Report final status
        ansible.builtin.debug:
          msg: "Deploy on {{ ansible_hostname }}: check complete"
```

4. Delegation — delegate_to

delegate_to runs a task on a different host than the current play target. Essential for load balancer drain, monitoring registration, and centralised operations.

```
tasks:

  # Remove from load balancer BEFORE updating
  - name: Disable server in HAProxy
    community.general.haproxy:
      state: disabled
      host: "{{ ansible_hostname }}"
      backend: web_pool
    delegate_to: lb01 # run on the load balancer

  # Update the web server
  - name: Update application
    ansible.builtin.dnf:
      name: myapp
      state: latest

  # Re-enable in load balancer
  - name: Enable server in HAProxy
    community.general.haproxy:
      state: enabled
      host: "{{ ansible_hostname }}"
      backend: web_pool
    delegate_to: lb01

  # Run a task on the control node itself
  - name: Log deployment to control node
    ansible.builtin.lineinfile:
      path: /var/log/deployments.log
      line: "{{ ansible_date_time.iso8601 }} {{ ansible_hostname }} updated"
```

```
        create: true
      delegate_to: localhost
```

5. serial — Rolling Updates

```
---
- name: Rolling update across web servers
  hosts: webservers
  become: true
  serial: 1 # update one at a time
  # serial: "30%" # update 30% of hosts at a time
  # serial: # staged rollout
  # - 1 # first update 1 host
  # - 30% # then 30%
  # - 100% # then the rest

  max_fail_percentage: 20 # abort if more than 20% of hosts fail

  tasks:

    - name: Update and verify
      ansible.builtin.dnf:
        name: nginx
        state: latest
      notify: Restart nginx

    - name: Flush handlers
      ansible.builtin.meta: flush_handlers

    - name: Verify after update
      ansible.builtin.uri:
        url: "http://{{ ansible_default_ipv4.address }}"
        status_code: 200
      retries: 5
      delay: 3

  handlers:
    - name: Restart nginx
```

```
      ansible.builtin.service:
        name: nginx
        state: restarted
```

6. wait_for — Wait for Conditions

```
  tasks:

    # Wait for a port to open
    - name: Wait for nginx to start listening
      ansible.builtin.wait_for:
        port: 80
        host: "{{ ansible_default_ipv4.address }}"
        delay: 2
        timeout: 30

    # Wait for SSH after reboot
    - name: Reboot and wait for SSH
      ansible.builtin.reboot:
        reboot_timeout: 180

    # Wait for a string in a log file
    - name: Wait for app startup in logs
      ansible.builtin.wait_for:
        path: /var/log/myapp/app.log
        search_regex: "Server started on port"
        timeout: 120

    # Wait for a file to disappear (lock file)
    - name: Wait for backup to complete
      ansible.builtin.wait_for:
        path: /var/run/backup.lock
        state: absent
        timeout: 3600
```

7. Async — Long-Running Tasks

```
  tasks:
```

```yaml
    # Start a long task and continue
    - name: Start long-running backup (async)
      ansible.builtin.shell: /opt/scripts/full_backup.sh
      async: 3600 # task allowed to run up to 1 hour
      poll: 0 # don't wait — fire and forget
      register: backup_job

    # Do other things while backup runs
    - name: Update packages while backup runs
      ansible.builtin.dnf:
        name: "*"
        state: latest

    # Check on the async task
    - name: Wait for backup to complete
      ansible.builtin.async_status:
        jid: "{{ backup_job.ansible_job_id }}"
      register: backup_result
      until: backup_result.finished
      retries: 60
      delay: 60 # check every minute
```

8. Lookup Plugins

```yaml
  tasks:

    # Read a file from control node
    - name: Deploy SSH public key
      ansible.posix.authorized_key:
        user: ansible
        key: "{{ lookup('file', '~/.ssh/id_ed25519.pub') }}"

    # Read env variable from control node
    - name: Use environment variable
      ansible.builtin.debug:
        msg: "Home dir: {{ lookup('env', 'HOME') }}"
```

```
    # Generate a random password
    - name: Set random password for user
      ansible.builtin.user:
        name: tempuser
        password: >-
          {{ lookup('password', '/tmp/tempuser_pass
                    length=16
                    chars=ascii_letters,digits')
            | password_hash('sha512') }}"
```

9. Real-World — Production Deployment Playbook

```
# production_deploy.yml
---
- name: Production deployment with vault, tags, error handling
  hosts: webservers
  become: true
  serial: "50%"
  max_fail_percentage: 0

  vars_files:
    - group_vars/all/vault.yml

  vars:
    app_name: mywebapp
    app_version: "{{ deploy_version | default('latest') }}"
    app_port: 8080

  tasks:

    # Pre-flight checks
    - name: Pre-flight | Check disk space
      ansible.builtin.shell: df /opt | awk 'NR==2{print $5}' | tr -d '%'
      register: disk_check
      changed_when: false
      failed_when: disk_check.stdout | int > 85
      tags: [preflight, always]
```

```yaml
# Deploy with full rollback support
- name: Deploy application
  tags: deploy
  block:

    - name: Deploy | Stop application
      ansible.builtin.service:
        name: "{{ app_name }}"
        state: stopped

    - name: Deploy | Install new version
      ansible.builtin.dnf:
        name: "{{ app_name }}-{{ app_version }}"
        state: present

    - name: Deploy | Deploy config with secrets
      ansible.builtin.template:
        src: templates/app.conf.j2
        dest: /etc/{{ app_name }}/app.conf
        mode: '0600'
      vars:
        db_pass: "{{ vault_db_password }}"

    - name: Deploy | Start application
      ansible.builtin.service:
        name: "{{ app_name }}"
        state: started

    - name: Deploy | Wait for app to respond
      ansible.builtin.wait_for:
        port: "{{ app_port }}"
        timeout: 60

    - name: Deploy | Health check
      ansible.builtin.uri:
        url: "http://localhost:{{ app_port }}/health"
```

```
            status_code: 200
          retries: 5
          delay: 5

      rescue:
        - name: ROLLBACK | Restore from backup
          ansible.builtin.shell: |
            BACKUP=$(ls -t /opt/backups/*.tar.gz | head -1)
            tar -xzf "$BACKUP" -C /

        - name: ROLLBACK | Fail after rollback
          ansible.builtin.fail:
            msg: "Deployment failed and rollback applied."

      always:
        - name: LOG | Record deployment attempt
          ansible.builtin.lineinfile:
            path: /var/log/deployments.log
            line: "{{ ansible_date_time.iso8601 }} {{ ansible_hostname }} {{ app_version
}}"
            create: true
          delegate_to: localhost
```

■ Key Takeaways

- Vault — always encrypt secrets with ansible-vault encrypt; use vault_ prefix; run with --vault-password-file in CI/CD
- Tags — tag every task; use always for critical checks, never for optional/destructive tasks
- block/rescue/always — try/except/finally — use for any task sequence that needs rollback capability
- delegate_to — run a task on a different host; essential for infrastructure coordination
- serial — controls rolling update batch size; max_fail_percentage aborts if too many hosts fail
- wait_for — gate on port, file, or string before proceeding — essential after reboots and service starts
- async — fire long-running tasks in background, check with async_status
- no_log: true — always set on tasks that handle passwords or secrets
- lookup() — pull data from files, env vars, CSV, password generators into playbooks

■ Module 7 Exercise — secure_deploy.yml

Write a playbook secure_deploy.yml that demonstrates all Module 7 concepts:

- Vault — create vault.yml with vault_app_password and vault_api_key; reference via vars.yml
- Tags — tag all tasks with setup, deploy, verify, notify; tag health check with never
- block/rescue/always — wrap main deployment; rescue logs and cleans up; always reports final status
- Error handling — failed_when on disk space check (fail if > 85%); ignore_errors on stopping service
- delegate_to: localhost — write deployment log to control node after each host
- wait_for — after starting service, wait for port 80 with 30-second timeout

- serial: 1 — deploy one host at a time across webservers
- no_log: true — on any task that uses vault_app_password

Module 7 Solution — secure_deploy.yml

```yaml
# group_vars/all/vault.yml — ENCRYPTED (content before encryption)
---
vault_app_password: "SuperSecretAppPass123!"
vault_api_key: "xK9mN2pQ8rT5vW3yA6bD1cE4fG7hJ0"

# group_vars/all/vars.yml — plain text references
---
app_password: "{{ vault_app_password }}"
api_key: "{{ vault_api_key }}"
app_name: mywebapp
app_port: 80
app_user: nginx
deploy_dir: /opt/mywebapp
```

```yaml
# secure_deploy.yml
---
- name: Secure application deployment
  hosts: webservers
  become: true
  serial: 1 # one host at a time — rolling deploy
  max_fail_percentage: 0 # abort entire play if any host fails

  tasks:

    # SETUP — pre-flight checks
    - name: Setup | Check disk space on /opt
      ansible.builtin.shell:
        cmd: df /opt | awk 'NR==2{print $5}' | tr -d '%'
      register: disk_usage
      changed_when: false
      failed_when: disk_usage.stdout | int > 85
      tags:
        - setup
```

```yaml
      - always # always run

    - name: Setup | Show disk usage
      ansible.builtin.debug:
        msg: "Disk {{ ansible_hostname }}: {{ disk_usage.stdout }}% (limit: 85%)"
      tags: [setup, always]

    - name: Setup | Create required directories
      ansible.builtin.file:
        path: "{{ item }}"
        state: directory
        mode: '0755'
      loop:
        - "{{ deploy_dir }}"
        - /var/log/mywebapp
      tags: setup

    - name: Setup | Stop existing service
      ansible.builtin.service:
        name: nginx
        state: stopped
      ignore_errors: true # might not be running yet
      tags: setup

    # DEPLOY — main deployment with rollback
    - name: Deploy application
      tags: deploy
      block:

        - name: Deploy | Write application config
          ansible.builtin.template:
            src: templates/app_config.j2
            dest: "{{ deploy_dir }}/app.conf"
            mode: '0600'
          no_log: true # contains vault vars

        - name: Deploy | Deploy index page
```

```
      ansible.builtin.copy:
        content: |
          <h1>{{ app_name }} on {{ ansible_hostname }}</h1>
          <p>Deployed: {{ ansible_date_time.iso8601 }}</p>
        dest: /usr/share/nginx/html/index.html
        mode: '0644'

    - name: Deploy | Start nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true

    - name: Deploy | Wait for port 80 to open
      ansible.builtin.wait_for:
        host: "{{ ansible_default_ipv4.address }}"
        port: "{{ app_port }}"
        delay: 2
        timeout: 30

  rescue:
    - name: RESCUE | Log failure details
      ansible.builtin.debug:
        msg: "DEPLOYMENT FAILED on {{ ansible_hostname }} — cleaning up"

    - name: RESCUE | Stop partial service
      ansible.builtin.service:
        name: nginx
        state: stopped
      ignore_errors: true

    - name: RESCUE | Write failure to control node
      ansible.builtin.lineinfile:
        path: /tmp/deploy_failures.log
        line: "{{ ansible_date_time.iso8601 }} FAILED {{ ansible_hostname }}"
        create: true
```

```
          delegate_to: localhost

        - name: RESCUE | Fail with clear message
          ansible.builtin.fail:
            msg: "Deployment failed on {{ ansible_hostname }}. Check
/tmp/deploy_failures.log"

      always:
        - name: ALWAYS | Check service status
          ansible.builtin.shell: systemctl is-active nginx
          register: nginx_status
          changed_when: false
          failed_when: false

        - name: ALWAYS | Report outcome
          ansible.builtin.debug:
            msg: "{{ ansible_hostname }}: nginx={{ nginx_status.stdout | upper }} disk={{
disk_usage.stdout }}%"

        - name: ALWAYS | Write deployment log to control node
          ansible.builtin.lineinfile:
            path: /tmp/deployments.log
            line: "{{ ansible_date_time.iso8601 }} {{ ansible_hostname }} nginx={{
nginx_status.stdout }}"
            create: true
          delegate_to: localhost

    # VERIFY — health checks (opt-in with --tags verify)
    - name: Verify | HTTP health check
      ansible.builtin.uri:
        url: "http://{{ ansible_default_ipv4.address }}"
        status_code: 200
        timeout: 10
      register: health_result
      retries: 3
      delay: 5
      tags:
```

```yaml
      - verify
      - never

  - name: Verify | Report result
    ansible.builtin.debug:
      msg: >-
        Health on {{ ansible_hostname }}:
        {{ 'PASSING' if health_result.status | default(0) == 200 else 'FAILING' }}
    tags: [verify, never]

  # NOTIFY
  - name: Notify | Send notification
    ansible.builtin.debug:
      msg: "{{ app_name }} deployed to {{ ansible_hostname }} successfully"
    delegate_to: localhost
    tags: notify
```

■ serial: 1 with max_fail_percentage: 0 gives safe rolling deployments — a bad deploy on web01 stops before web02 and web03 are touched. The rescue block always ends with ansible.builtin.fail to re-raise after cleanup, so the play recap shows failed=1 and CI/CD catches it.

Module 8 RHCE Exam Projects

5 complete exam-style automation scenarios — EX294 ready

How the RHCE Exam Works

- 4 hours, hands-on practical exam — no multiple choice
- You get a control node and managed nodes (RHEL 9)
- Tasks are graded by running your playbooks and checking the resulting state
- Partial credit — a playbook that does 3 of 5 things correctly gets partial marks
- ansible-playbook --check is your best friend before submitting

```
# Exam survival checklist — run before every task:

ansible all -m ping # confirm connectivity

ansible-playbook task.yml --syntax-check # catch YAML errors

ansible-playbook task.yml --list-tasks # confirm task list

ansible-playbook task.yml --check # dry run

ansible-playbook task.yml # apply

ansible all -m shell -a "systemctl status nginx" # verify result
```

Project 1 — System Configuration at Scale

Requirement: Write system_config.yml that runs on all managed nodes and:

- Installs tmux, vim, wget, curl, net-tools
- Sets the timezone to Europe/London
- Ensures chronyd is started and enabled
- Creates group sysops with GID 4000
- Creates user sysops with UID 4000, primary group sysops, home /home/sysops, shell /bin/bash
- Adds sysops to the wheel group
- Deploys /etc/motd from a template showing hostname, OS, and RAM
- Ensures sshd is running and enabled
- Opens port 22 permanently in firewalld

Solution — system_config.yml

```
# system_config.yml

---

- name: System configuration for all managed nodes

  hosts: all

  become: true


  vars:

    required_packages: [tmux, vim, wget, curl, net-tools]

    sysops_uid: 4000
```

```yaml
    sysops_gid: 4000

  tasks:

    - name: Install required packages
      ansible.builtin.dnf:
        name: "{{ required_packages }}"
        state: present
      tags: packages

    - name: Set timezone to Europe/London
      community.general.timezone:
        name: Europe/London
      notify: Restart chronyd
      tags: system

    - name: Start and enable chronyd
      ansible.builtin.service:
        name: chronyd
        state: started
        enabled: true
      tags: system

    - name: Create sysops group
      ansible.builtin.group:
        name: sysops
        gid: "{{ sysops_gid }}"
        state: present
      tags: users

    - name: Create sysops user
      ansible.builtin.user:
        name: sysops
        uid: "{{ sysops_uid }}"
        group: sysops
        groups: wheel
        append: true
```

```
        home: /home/sysops
        shell: /bin/bash
        create_home: true
        state: present
      tags: users

    - name: Deploy MOTD from template
      ansible.builtin.copy:
        content: |
          ================================================
           Host : {{ ansible_hostname }}
           OS : {{ ansible_distribution }} {{ ansible_distribution_version }}
           RAM : {{ (ansible_memtotal_mb / 1024) | round(1) }} GB
          ================================================
        dest: /etc/motd
        mode: '0644'
      tags: motd

    - name: Ensure sshd is started and enabled
      ansible.builtin.service:
        name: sshd
        state: started
        enabled: true

    - name: Open SSH port in firewalld
      ansible.builtin.firewalld:
        service: ssh
        state: enabled
        permanent: true
        immediate: true
      tags: firewall

  handlers:
    - name: Restart chronyd
      ansible.builtin.service:
```

```
        name: chronyd
        state: restarted
```

Project 2 — Web Infrastructure Deployment

Requirement: Write web_infrastructure.yml with two plays:

Play 1 targets webservers: Install nginx and php-fpm, deploy nginx.conf with server_tokens off, create index.php, start services, open ports 80/443, verify with uri module.

Play 2 targets databases: Install mariadb-server, start and enable, open port 3306, set root password from vault.

```
# web_infrastructure.yml
---
# Play 1: Web Servers
- name: Configure web servers
  hosts: webservers
  become: true

  vars:
    http_port: 80

  tasks:

    - name: Install nginx and php-fpm
      ansible.builtin.dnf:
        name: [nginx, php-fpm, php]
        state: present

    - name: Deploy nginx.conf from template
      ansible.builtin.copy:
        content: |
          user nginx;
          worker_processes auto;
          pid /run/nginx.pid;
          events { worker_connections 1024; }
          http {
              server_tokens off;
              include /etc/nginx/mime.types;
              server {
```

```
                listen {{ http_port }};
                server_name {{ ansible_fqdn }};
                root /var/www/html;
                index index.php index.html;
                location ~ \.php$ {
                    fastcgi_pass unix:/run/php-fpm/www.sock;
                    fastcgi_index index.php;
                    include fastcgi_params;
                    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
                }
            }
        }
      dest: /etc/nginx/nginx.conf
      validate: nginx -t -c %s
    notify: Restart nginx

  - name: Create web root
    ansible.builtin.file:
      path: /var/www/html
      state: directory
      owner: nginx
      group: nginx
      mode: '0755'

  - name: Deploy index.php
    ansible.builtin.copy:
      content: '<?php echo "Server: " . gethostname(); ?>'
      dest: /var/www/html/index.php
      owner: nginx
      group: nginx
      mode: '0644'

  - name: Start and enable nginx
    ansible.builtin.service:
      name: nginx
```

```yaml
        state: started
        enabled: true

    - name: Start and enable php-fpm
      ansible.builtin.service:
        name: php-fpm
        state: started
        enabled: true

    - name: Open HTTP and HTTPS in firewalld
      ansible.builtin.firewalld:
        service: "{{ item }}"
        state: enabled
        permanent: true
        immediate: true
      loop: [http, https]

    - name: Verify nginx responds on port 80
      ansible.builtin.uri:
        url: "http://{{ ansible_default_ipv4.address }}"
        status_code: 200
      register: nginx_check
      retries: 3
      delay: 3

  handlers:
    - name: Restart nginx
      ansible.builtin.service:
        name: nginx
        state: restarted

# Play 2: Database Servers
- name: Configure database servers
  hosts: databases
  become: true

  tasks:
```

```yaml
- name: Install MariaDB
  ansible.builtin.dnf:
    name: [mariadb-server, mariadb, python3-PyMySQL]
    state: present

- name: Start and enable MariaDB
  ansible.builtin.service:
    name: mariadb
    state: started
    enabled: true

- name: Set MariaDB root password
  community.mysql.mysql_user:
    name: root
    host_all: true
    password: "{{ vault_db_root_password }}"
    login_unix_socket: /var/lib/mysql/mysql.sock
  no_log: true

- name: Remove anonymous MySQL users
  community.mysql.mysql_user:
    name: ''
    host_all: true
    state: absent
    login_user: root
    login_password: "{{ vault_db_root_password }}"
    login_unix_socket: /var/lib/mysql/mysql.sock
  no_log: true

- name: Remove test database
  community.mysql.mysql_db:
    name: test
    state: absent
    login_user: root
    login_password: "{{ vault_db_root_password }}"
```

```
        login_unix_socket: /var/lib/mysql/mysql.sock
      no_log: true

    - name: Open MySQL port in firewalld
      ansible.builtin.firewalld:
        port: 3306/tcp
        state: enabled
        permanent: true
        immediate: true
```

Project 3 — Role-Based User Management

```
# roles/user_management/defaults/main.yml
---
users: []
default_shell: /bin/bash
default_groups: []
sudo_enabled: false

# roles/user_management/tasks/main.yml
---
- name: Manage user accounts
  ansible.builtin.user:
    name: "{{ item.name }}"
    uid: "{{ item.uid | default(omit) }}"
    comment: "{{ item.comment | default(omit) }}"
    shell: "{{ item.shell | default(default_shell) }}"
    home: "{{ item.home | default('/home/' + item.name) }}"
    groups: >-
      {{ ['wheel'] + default_groups
         if item.sudo | default(sudo_enabled) | bool
         else default_groups }}
    append: true
    create_home: true
    state: "{{ item.state | default('present') }}"
  loop: "{{ users }}"
```

```
    loop_control:
      label: "{{ item.name }}"

- name: Deploy SSH authorized keys
    ansible.posix.authorized_key:
      user: "{{ item.name }}"
      key: "{{ item.ssh_key }}"
      state: present
    loop: "{{ users }}"
    loop_control:
      label: "{{ item.name }}"
    when:
      - item.ssh_key is defined
      - item.state | default('present') == 'present'
```

```
# manage_users.yml
---
- name: Role-based user management across all hosts
    hosts: all
    become: true

    vars:
      users:
        - name: alice
          uid: 3001
          comment: "Alice — Developer"
          sudo: true

        - name: bob
          uid: 3002
          comment: "Bob — Developer"
          sudo: true
          ssh_key: "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAA bob@workstation"

        - name: charlie
          uid: 3003
```

```
        comment: "Charlie — Junior Dev"
        sudo: false

      - name: olduser
        uid: 3099
        state: absent # will be removed

  roles:
    - role: user_management
```

Project 4 — Automated Security Hardening

```
# harden.yml
---
- name: Security hardening for all managed hosts
  hosts: all
  become: true

  vars:
    hardened_sysctl:
      net.ipv4.ip_forward: 0
      net.ipv4.conf.all.accept_redirects: 0
      net.ipv4.conf.all.send_redirects: 0
      kernel.randomize_va_space: 2
      net.ipv4.tcp_syncookies: 1

    unwanted_services:
      - bluetooth
      - cups
      - avahi-daemon

  tasks:

    # SELinux
    - name: SELinux | Set enforcing mode
      ansible.posix.selinux:
        state: enforcing
        policy: targeted
```

```yaml
    register: selinux_result
    tags: selinux

  - name: SELinux | Reboot if mode changed
    ansible.builtin.reboot:
      reboot_timeout: 120
    when: selinux_result.reboot_required | default(false)
    tags: selinux

  # Disable unwanted services
  - name: Services | Disable and mask unwanted services
    ansible.builtin.systemd:
      name: "{{ item }}"
      state: stopped
      enabled: false
      masked: true
    loop: "{{ unwanted_services }}"
    ignore_errors: true
    tags: services

  # Kernel parameters
  - name: Sysctl | Apply hardened kernel parameters
    ansible.posix.sysctl:
      name: "{{ item.key }}"
      value: "{{ item.value }}"
      state: present
      reload: true
      sysctl_file: /etc/sysctl.d/99-hardening.conf
    loop: "{{ hardened_sysctl | dict2items }}"
    loop_control:
      label: "{{ item.key }} = {{ item.value }}"
    tags: sysctl

  # SSH hardening
  - name: SSH | Deploy hardened sshd_config
```

```yaml
    ansible.builtin.copy:
      content: |
        Protocol 2
        PermitRootLogin no
        PasswordAuthentication no
        PubkeyAuthentication yes
        MaxAuthTries 3
        MaxSessions 5
        ClientAliveInterval 300
        ClientAliveCountMax 2
        X11Forwarding no
        Banner /etc/issue.net
        Subsystem sftp /usr/lib/openssh/sftp-server
      dest: /etc/ssh/sshd_config
      mode: '0600'
      validate: sshd -t -f %s
      backup: true
    notify: Restart sshd
    tags: ssh

  # Firewall
  - name: Firewall | Start and enable firewalld
    ansible.builtin.service:
      name: firewalld
      state: started
      enabled: true
    tags: firewall

  - name: Firewall | Set default zone to drop
    ansible.builtin.command:
      cmd: firewall-cmd --set-default-zone=drop
    register: zone_result
    changed_when: "'success' in zone_result.stdout"
    tags: firewall
```

```
    - name: Firewall | Allow SSH in drop zone
      ansible.builtin.firewalld:
        zone: drop
        service: ssh
        state: enabled
        permanent: true
        immediate: true
      tags: firewall

    # TCP wrappers
    - name: Hosts deny | Block all by default
      ansible.builtin.lineinfile:
        path: /etc/hosts.deny
        line: "ALL: ALL"
        state: present
        create: true
      tags: tcpwrappers

    - name: Hosts allow | Permit control node
      ansible.builtin.lineinfile:
        path: /etc/hosts.allow
        line: "ALL: {{ hostvars['control'].ansible_default_ipv4.address }}"
        state: present
        create: true
      tags: tcpwrappers

  handlers:
    - name: Restart sshd
      ansible.builtin.service:
        name: sshd
        state: restarted
```

Project 5 — Complete Multi-Tier Application Stack

A complete end-to-end deployment using all modules: baseline role, nginx, mariadb, PHP app connecting to database, serial rolling updates, post-deploy verification, vault for credentials.

```
# full_stack.yml
```

```yaml
# Run: ansible-playbook full_stack.yml --vault-password-file ~/.vault_pass
---

# Play 1: Baseline all hosts
- name: Apply baseline to all hosts
  hosts: all
  become: true
  tags: baseline
  roles:
    - role: baseline
      vars:
        baseline_timezone: UTC

# Play 2: Database server
- name: Configure database server
  hosts: databases
  become: true
  tags: db

  tasks:

    - name: DB | Install MariaDB stack
      ansible.builtin.dnf:
        name: [mariadb-server, mariadb, python3-PyMySQL]
        state: present

    - name: DB | Start and enable MariaDB
      ansible.builtin.service:
        name: mariadb
        state: started
        enabled: true

    - name: DB | Set root password
      community.mysql.mysql_user:
        name: root
        host_all: true
        password: "{{ vault_db_root_password }}"
```

```
        login_unix_socket: /var/lib/mysql/mysql.sock
      no_log: true

    - name: DB | Create application database
      community.mysql.mysql_db:
        name: "{{ app_db_name | default('webapp') }}"
        state: present
        login_user: root
        login_password: "{{ vault_db_root_password }}"
        login_unix_socket: /var/lib/mysql/mysql.sock
      no_log: true

    - name: DB | Create application user
      community.mysql.mysql_user:
        name: "{{ app_db_user | default('webuser') }}"
        password: "{{ vault_db_password }}"
        priv: "{{ app_db_name | default('webapp') }}.*:ALL"
        host: "{{ item }}"
        state: present
        login_user: root
        login_password: "{{ vault_db_root_password }}"
        login_unix_socket: /var/lib/mysql/mysql.sock
      loop: "{{ groups['webservers']
                | map('extract', hostvars,
                        ['ansible_default_ipv4', 'address'])
                | list }}"
      no_log: true

    - name: DB | Open MySQL port for webservers
      ansible.builtin.firewalld:
        rich_rule: >-
          rule family="ipv4"
          source address="{{ item }}/32"
          port port="3306" protocol="tcp" accept
        state: enabled
```

```
        permanent: true
        immediate: true
      loop: "{{ groups['webservers']
               | map('extract', hostvars,
                     ['ansible_default_ipv4', 'address'])
               | list }}"

# Play 3: Web servers — rolling deploy
- name: Configure web servers
  hosts: webservers
  become: true
  serial: 1
  tags: web

  tasks:

    - name: Web | Install nginx and php-fpm
      ansible.builtin.dnf:
        name: [nginx, php-fpm, php, php-mysqlnd]
        state: present

    - name: Web | Deploy PHP application
      ansible.builtin.copy:
        content: |
          <?php
          $db_host = "{{ hostvars[groups['databases'][0]].ansible_default_ipv4.address
}}";
          $db_name = "{{ app_db_name | default('webapp') }}";
          $db_user = "{{ app_db_user | default('webuser') }}";
          $db_pass = "{{ vault_db_password }}";
          $conn = new mysqli($db_host, $db_user, $db_pass, $db_name);
          header('Content-Type: application/json');
          echo json_encode([
              "server" => gethostname(),
              "db_host" => $db_host,
              "db_status" => $conn->connect_error ? "ERROR" : "OK",
```

```
                "deployed" => "{{ ansible_date_time.iso8601 }}"
            ]);
            ?>
        dest: /var/www/html/index.php
        owner: nginx
        group: nginx
        mode: '0644'
      no_log: true # contains vault_db_password

    - name: Web | Start and enable nginx + php-fpm
      ansible.builtin.service:
        name: "{{ item }}"
        state: started
        enabled: true
      loop: [nginx, php-fpm]

    - name: Web | Open HTTP in firewall
      ansible.builtin.firewalld:
        service: http
        state: enabled
        permanent: true
        immediate: true

    - name: Web | Wait for nginx on port 80
      ansible.builtin.wait_for:
        host: "{{ ansible_default_ipv4.address }}"
        port: 80
        timeout: 30

  handlers:
    - name: Restart nginx
      ansible.builtin.service:
        name: nginx
        state: restarted

# Play 4: Post-deploy verification
```

```yaml
- name: Post-deploy verification
  hosts: webservers
  become: false
  tags:
    - verify
    - never # run explicitly: --tags verify

  tasks:

    - name: Verify | HTTP health check
      ansible.builtin.uri:
        url: "http://{{ ansible_default_ipv4.address }}/index.php"
        status_code: 200
        timeout: 10
      register: app_check
      ignore_errors: true

    - name: Verify | Report per-host result
      ansible.builtin.debug:
        msg: >-
          {{ ansible_hostname }}:
          {{ 'PASS — HTTP ' + (app_check.status | string)
             if app_check.status is defined and app_check.status == 200
             else 'FAIL — ' + (app_check.msg | default('no response')) }}

    - name: Verify | Write verification report
      ansible.builtin.lineinfile:
        path: /tmp/stack_verify.log
        line: >-
          {{ ansible_date_time.iso8601 }}
          {{ ansible_hostname }}
          {{ 'PASS' if app_check.status | default(0) == 200 else 'FAIL' }}
        create: true
      delegate_to: localhost

    - name: Verify | Fail if any server not responding
```

```
      ansible.builtin.fail:

        msg: "{{ ansible_hostname }} failed health check. HTTP: {{ app_check.status |
default('no response') }}"

      when: app_check.status | default(0) != 200
```

RHCE Exam Final Checklist

```
Before every task on exam day:

ansible all -m ping # confirm connectivity

ansible-playbook task.yml --syntax-check # catch YAML errors

ansible-playbook task.yml --list-tasks # confirm task list

ansible-playbook task.yml --check # dry run

ansible-playbook task.yml # apply

ansible all -m shell -a "systemctl status X" # verify result
```

Checklist Item	Why It Matters
ansible.cfg set up — inventory, remote_user, become	Without this, every command needs extra flags
SSH key-based auth working to all nodes	Ansible cannot prompt for passwords mid-run
All playbooks start with ---	Required YAML document start marker
YAML indentation is 2 spaces — no tabs anywhere	Tab characters cause cryptic parse errors
Module names use FQCN: ansible.builtin.dnf	Required in modern Ansible
service: always set both state: AND enabled:	state without enabled won't survive a reboot
firewalld: permanent: true AND immediate: true	Missing one = change lost or not applied
user: always use append: true with groups:	Without it, existing groups are removed
command/shell: use creates: or changed_when:	Makes commands idempotent
Vault file encrypted: ansible-vault view to confirm	Plain text vault = zero marks for that task
--check passes clean before final apply	Catches errors before they break things
Post-apply verification with ad-hoc commands	Proves to examiner (and yourself) it worked

Course Complete — What You Have Mastered

Module	Topic	Key Skills
1	Foundations	Inventory, ad-hoc, YAML, first playbook
2	Playbooks	Variables, handlers, when, loop, register
3	Modules	dnf, service, file, copy, template, user, firewalld
4	Variables & Facts	Facts, set_fact, hostvars, group_vars, filters
5	Roles	Role structure, defaults vs vars, Galaxy, site.yml
6	Jinja2 Templates	Loops, conditionals, macros, filters, real configs
7	Advanced Topics	Vault, tags, block/rescue, delegate_to, serial, async
8	RHCE Projects	5 exam-style end-to-end automation scenarios

Technology	Status
Bash Scripting	Complete
Python for SysAdmins	Complete
Ansible / RHCE	Complete
Terraform	Next
Docker	Upcoming
Kubernetes	Upcoming
CI/CD (Jenkins/GitHub Actions)	Upcoming
Cloud (AWS/Azure)	Upcoming